Generative AI Use: For the purposes of the assignments, the use of generative AI is subject to the same policies regarding collaboration. Just as with other collaborators, each student must write down the solutions independently of the output of the interaction and the submission should include a note denoting the nature of the collaboration. The use of generative AI tools to substantially complete sections of the assignments is not in line with the spirit of the assignments, and would be a violation of the Honor Code.

In [4]:
```
from pathlib import Path

work_dir = Path.cwd()
print("当前文件夹：", work_dir)
print("能否找到 cs231n：", (work_dir / "cs231n").is_dir())
data_dir = work_dir / "cs231n" / "datasets"
print("数据目录是否存在：", data_dir.is_dir())
print("目录内容：", [p.name for p in data_dir.iterdir()] if data_dir.is_dir() els
script_path = data_dir / "get_datasets.sh"
print(script_path.read_text(encoding="utf-8"))
import urllib.request
import tarfile

url = "https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz"
archive_path = data_dir / "cifar-10-python.tar.gz"

if (data_dir / "cifar-10-batches-py").is_dir():
    print("CIFAR-10 已存在，无需重复下载。")
else:
    print("正在下载，请稍等……")
    urllib.request.urlretrieve(url, archive_path)

    print("下载完成，正在解压……")
    with tarfile.open(archive_path, "r:gz") as archive:
        archive.extractall(path=data_dir, filter="data")

    print("解压完成。")
```

```
当前文件夹： E:\CS231n\assignment1
能否找到 cs231n： True
数据目录是否存在： True
目录内容： ['get_datasets.sh']
# cd cs231n/datasets
if [ ! -d "cifar-10-batches-py" ]; then
  wget http://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz -O cifar-10-python.
tar.gz
  tar -xzvf cifar-10-python.tar.gz
  rm cifar-10-python.tar.gz
  wget http://cs231n.stanford.edu/imagenet_val_25.npz
fi
# cd ../..
正在下载，请稍等……
下载完成，正在解压……
解压完成。
```

k-Nearest Neighbor (kNN) exercise

Complete and hand in this completed worksheet (including its outputs and any supporting code outside of the worksheet) with your assignment submission. For more details see the assignments page on the course website.

The kNN classifier consists of two stages:

- During training, the classifier takes the training data and simply remembers it
- During testing, kNN classifies every test image by comparing to all training images and transfering the labels of the k most similar training examples
- The value of k is cross-validated

In this exercise you will implement these steps and understand the basic Image Classification pipeline, cross-validation, and gain proficiency in writing efficient, vectorized code.

In [7]:
```
# Run some setup code for this notebook.

import random
import numpy as np
from cs231n.data_utils import load_CIFAR10
import matplotlib.pyplot as plt

# This is a bit of magic to make matplotlib figures appear inline in the noteboo
# rather than in a new window.
%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# Some more magic so that the notebook will reload external python modules;
# see http://stackoverflow.com/questions/1907993/autoreload-of-modules-in-ipytho
%load_ext autoreload
%autoreload 2
```

In [6]:
```
%pip install imageio
```

```
Collecting imageio
  Downloading imageio-2.38.1-py3-none-any.whl.metadata (9.7 kB)
Requirement already satisfied: numpy in .\.venv\Lib\site-packages (from imageio)
(2.5.3)
Requirement already satisfied: pillow>=8.3.2 in .\.venv\Lib\site-packages (from i
mageio) (12.3.0)
Downloading imageio-2.38.1-py3-none-any.whl (328 kB)
Installing collected packages: imageio
Successfully installed imageio-2.38.1
Note: you may need to restart the kernel to use updated packages.
```

In [8]:
```
# Load the raw CIFAR-10 data.
cifar10_dir = 'cs231n/datasets/cifar-10-batches-py'

# Cleaning up variables to prevent loading data multiple times (which may cause
try:
   del X_train, y_train
   del X_test, y_test
   print('Clear previously loaded data.')
except:
   pass
```

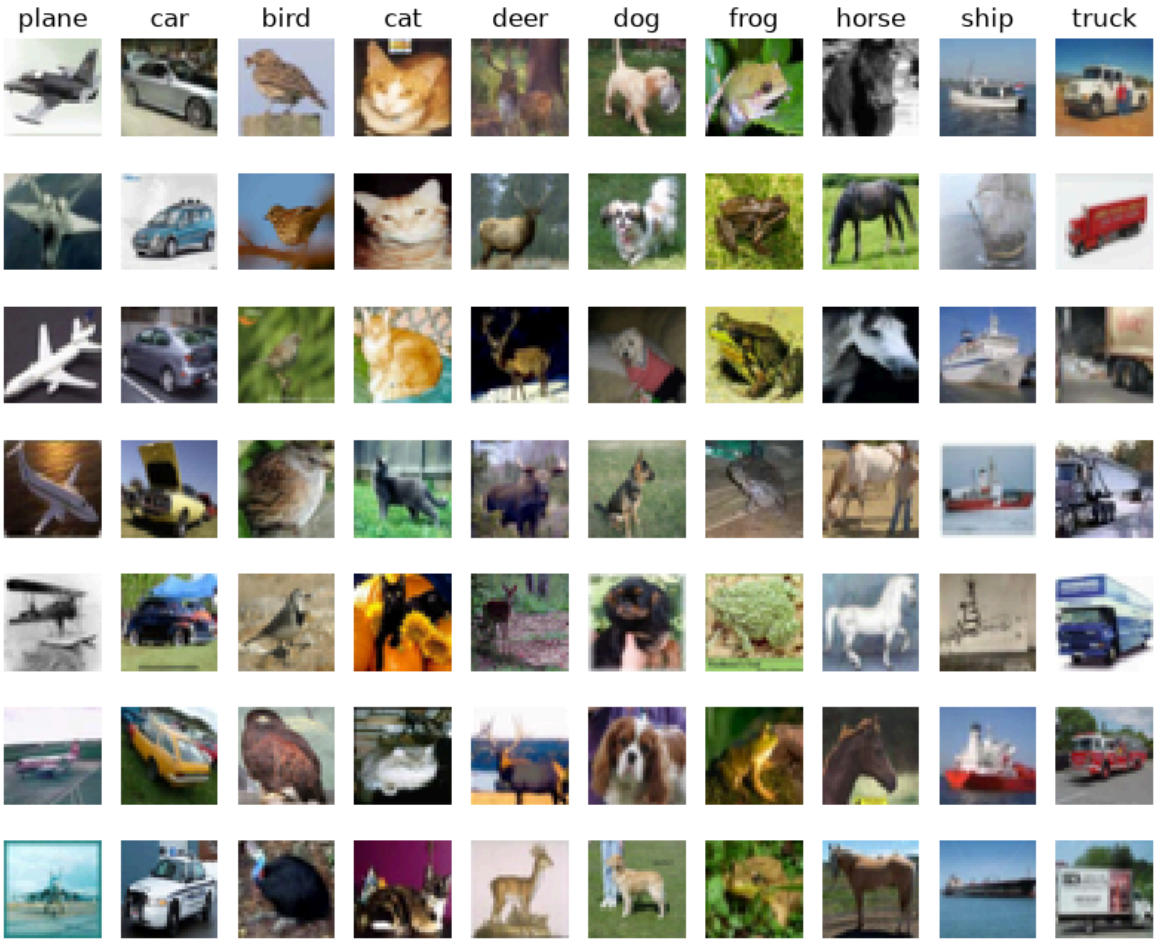
```
X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

# As a sanity check, we print out the size of the training and test data.
print('Training data shape: ', X_train.shape)
print('Training labels shape: ', y_train.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)
```

```
Training data shape:  (50000, 32, 32, 3)
Training labels shape:  (50000,)
Test data shape:  (10000, 32, 32, 3)
Test labels shape:  (10000,)
```

In [9]:

```
# Visualize some examples from the dataset.
# We show a few examples of training images from each class.
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship'
num_classes = len(classes)
samples_per_class = 7
for y, cls in enumerate(classes):
    idxs = np.flatnonzero(y_train == y)
    idxs = np.random.choice(idxs, samples_per_class, replace=False)
    for i, idx in enumerate(idxs):
        plt_idx = i * num_classes + y + 1
        plt.subplot(samples_per_class, num_classes, plt_idx)
        plt.imshow(X_train[idx].astype('uint8'))
        plt.axis('off')
        if i == 0:
            plt.title(cls)
plt.show()
```



In [10]:
```
# Subsample the data for more efficient code execution in this exercise
num_training = 5000
mask = list(range(num_training))
X_train = X_train[mask]
y_train = y_train[mask]

num_test = 500
mask = list(range(num_test))
X_test = X_test[mask]
y_test = y_test[mask]

# Reshape the image data into rows
X_train = np.reshape(X_train, (X_train.shape[0], -1))
X_test = np.reshape(X_test, (X_test.shape[0], -1))
print(X_train.shape, X_test.shape)
```

```
(5000, 3072) (500, 3072)
```

In [14]:
```
from cs231n.classifiers import KNearestNeighbor

# Create a kNN classifier instance.
# Remember that training a kNN classifier is a noop:
# the Classifier simply remembers the data and does no further processing
classifier = KNearestNeighbor()
classifier.train(X_train, y_train)
```

In [12]:
```
%pip install future
```

```
Collecting future
  Downloading future-1.0.0-py3-none-any.whl.metadata (4.0 kB)
Downloading future-1.0.0-py3-none-any.whl (491 kB)
Installing collected packages: future
Successfully installed future-1.0.0
Note: you may need to restart the kernel to use updated packages.
```

We would now like to classify the test data with the kNN classifier. Recall that we can break down this process into two steps:

1. First we must compute the distances between all test examples and all train examples.
2. Given these distances, for each test example we find the k nearest examples and have them vote for the label

Lets begin with computing the distance matrix between all training and test examples. For example, if there are **Ntr** training examples and **Nte** test examples, this stage should result in a **Nte x Ntr** matrix where each element (i,j) is the distance between the i-th test and j-th train example.

Note: For the three distance computations that we require you to implement in this notebook, you may not use the np.linalg.norm() function that numpy provides.

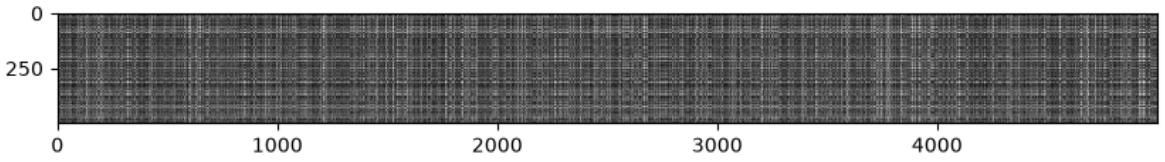
First, open `cs231n/classifiers/k_nearest_neighbor.py` and implement the function `compute_distances_two_loops` that uses a (very inefficient) double loop over all pairs of (test, train) examples and computes the distance matrix one element at a time.

In [15]:
```
# Open cs231n/classifiers/k_nearest_neighbor.py and implement
# compute_distances_two_loops.

# Test your implementation:
dists = classifier.compute_distances_two_loops(X_test)
print(dists.shape)
```

```
(500, 5000)
```

In [16]:
```
# We can visualize the distance matrix: each row is a single test example and
# its distances to training examples
plt.imshow(dists, interpolation='none')
plt.show()
```



Inline Question 1

Notice the structured patterns in the distance matrix, where some rows or columns are visibly brighter. (Note that with the default color scheme black indicates low distances while white indicates high distances.)

- What in the data is the cause behind the distinctly bright rows?
- What causes the columns?

$\color{blue}{\textit Your Answer:}$ *Bright rows correspond to test images whose pixel values differ substantially from those of most training images. Unusual overall brightness, colors, or backgrounds can make their L2 distances large.

Bright columns correspond to training images that differ substantially from most test images for the same reasons. Since the distances are computed from raw pixels, they are sensitive to these visual differences.*

In [20]:
```
# Now implement the function predict_labels and run the code below:
# We use k = 1 (which is Nearest Neighbor).
y_test_pred = classifier.predict_labels(dists, k=1)

# Compute and print the fraction of correctly predicted examples
num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
```

```
Got 137 / 500 correct => accuracy: 0.274000
```

You should expect to see approximately `27%` accuracy. Now lets try out a larger `k`, say `k = 5`:

In [21]:
```
y_test_pred = classifier.predict_labels(dists, k=5)
num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
```

```
Got 139 / 500 correct => accuracy: 0.278000
```

In [19]:
```
labels, counts = np.unique(y_test_pred, return_counts=True)
print("预测的类别：", labels)
print("每个类别的预测数量：", counts)
```

```
预测的类别： [0.]
每个类别的预测数量： [500]
```

You should expect to see a slightly better performance than with `k = 1`.

Inline Question 2

We can also use other distance metrics such as L1 distance. For pixel values $p_{ij}^{(k)}$ at location (i,j) of some image I_k,

the mean μ across all pixels over all images is $$\mu=\frac{1}{nhw}\sum_{k=1}^n\sum_{i=1}^{h}\sum_{j=1}^{w}p_{ij}^{(k)}$$ And the pixel-wise mean μ_{ij} across all images is $$\mu_{ij}=\frac{1}{n}\sum_{k=1}^np_{ij}^{(k)}.$$ The general standard deviation σ and pixel-wise standard deviation σ_{ij} is defined similarly.

Which of the following preprocessing steps will not change the performance of a Nearest Neighbor classifier that uses L1 distance? Select all that apply. To clarify, both training and test examples are preprocessed in the same way.

1. Subtracting the mean μ ($\tilde{p}_{ij}^{(k)}=p_{ij}^{(k)}-\mu$.)
2. Subtracting the per pixel mean μ_{ij} ($\tilde{p}_{ij}^{(k)}=p_{ij}^{(k)}-\mu_{ij}$.)
3. Subtracting the mean μ and dividing by the standard deviation σ.
4. Subtracting the pixel-wise mean μ_{ij} and dividing by the pixel-wise standard deviation σ_{ij}.
5. Rotating the coordinate axes of the data, which means rotating all the images by the same angle. Empty regions in the image caused by rotation are padded with a same pixel value and no interpolation is performed.

$\color{blue}{\textit Your Answer:1, 2, 3}$

$\color{blue}{\textit Your Explanation: 1 和 2：训练图片与测试图片在同一像素位置减去相同的均值，计算差值时均值会抵消，因此 L1 距离不变。

3：减去共同均值后，再除以同一个正标准差，只会将所有距离乘以同一个正比例，不改变最近邻的排序。

4：每个像素位置除以不同的标准差，会改变各位置对距离的贡献，可能改变最近邻的排序。

5：一般角度的图片旋转即使不插值，也可能造成像素丢失、重复或裁剪，因此不能保证 L1 距离和最近邻排序不变。}$

In [22]:
```
# Now lets speed up distance matrix computation by using partial vectorization
# with one loop. Implement the function compute_distances_one_loop and run the
# code below:
dists_one = classifier.compute_distances_one_loop(X_test)

# To ensure that our vectorized implementation is correct, we make sure that it
# agrees with the naive implementation. There are many ways to decide whether
# two matrices are similar; one of the simplest is the Frobenius norm. In case
# you haven't seen it before, the Frobenius norm of two matrices is the square
# root of the squared sum of differences of all elements; in other words, reshap
# the matrices into vectors and compute the Euclidean distance between them.
difference = np.linalg.norm(dists - dists_one, ord='fro')
print('One loop difference was: %f' % (difference, ))
if difference < 0.001:
    print('Good! The distance matrices are the same')
else:
    print('Uh-oh! The distance matrices are different')
```

```
One loop difference was: 0.000000
Good! The distance matrices are the same
```

In [23]:
```
# Now implement the fully vectorized version inside compute_distances_no_loops
# and run the code
dists_two = classifier.compute_distances_no_loops(X_test)

# check that the distance matrix agrees with the one we computed before:
difference = np.linalg.norm(dists - dists_two, ord='fro')
print('No loop difference was: %f' % (difference, ))
if difference < 0.001:
    print('Good! The distance matrices are the same')
else:
    print('Uh-oh! The distance matrices are different')
```

```
No loop difference was: 0.000000
Good! The distance matrices are the same
```

In [24]:
```
# Let's compare how fast the implementations are
def time_function(f, *args):
    """
    Call a function f with args and return the time (in seconds) that it took to
    """
    import time
    tic = time.time()
    f(*args)
    toc = time.time()
    return toc - tic

two_loop_time = time_function(classifier.compute_distances_two_loops, X_test)
print('Two loop version took %f seconds' % two_loop_time)

one_loop_time = time_function(classifier.compute_distances_one_loop, X_test)
print('One loop version took %f seconds' % one_loop_time)

no_loop_time = time_function(classifier.compute_distances_no_loops, X_test)
print('No loop version took %f seconds' % no_loop_time)

# You should see significantly faster performance with the fully vectorized impl

# NOTE: depending on what machine you're using,
```

```
# you might not see a speedup when you go from two loops to one loop,
# and might even see a slow-down.
```

```
Two loop version took 13.288631 seconds
One loop version took 32.599643 seconds
No loop version took 0.090188 seconds
```

Cross-validation

We have implemented the k-Nearest Neighbor classifier but we set the value k = 5 arbitrarily. We will now determine the best value of this hyperparameter with cross-validation.

In [26]:
```
num_folds = 5
k_choices = [1, 3, 5, 8, 10, 12, 15, 20, 50, 100]

X_train_folds = []
y_train_folds = []
################################################################################
# TODO:                                                                        #
# Split up the training data into folds. After splitting, X_train_folds and    #
# y_train_folds should each be lists of length num_folds, where                #
# y_train_folds[i] is the label vector for the points in X_train_folds[i].     #
# Hint: Look up the numpy array_split function.                                #
################################################################################
X_train_folds = np.array_split(X_train, num_folds, axis=0)
y_train_folds = np.array_split(y_train, num_folds, axis=0)
print("分成了几份：", len(X_train_folds), len(y_train_folds))
print("第一份图片形状：", X_train_folds[0].shape)
print("第一份标签形状：", y_train_folds[0].shape)

# A dictionary holding the accuracies for different values of k that we find
# when running cross-validation. After running cross-validation,
# k_to_accuracies[k] should be a list of length num_folds giving the different
# accuracy values that we found when using that value of k.
k_to_accuracies = {}


################################################################################
# TODO:                                                                        #
# Perform k-fold cross validation to find the best value of k. For each        #
# possible value of k, run the k-nearest-neighbor algorithm num_folds times,   #
# where in each case you use all but one of the folds as training data and the #
# last fold as a validation set. Store the accuracies for all fold and all     #
# values of k in the k_to_accuracies dictionary.                               #
################################################################################

for fold in range(num_folds):
    X_val = X_train_folds[fold]
    y_val = y_train_folds[fold]

    X_train_cv = np.concatenate(
        [X_train_folds[j] for j in range(num_folds) if j != fold],
        axis=0
    )
    y_train_cv = np.concatenate(
        [y_train_folds[j] for j in range(num_folds) if j != fold],
        axis=0
    )
```

```
    cv_classifier = KNearestNeighbor()
    cv_classifier.train(X_train_cv, y_train_cv)
    val_dists = cv_classifier.compute_distances_no_loops(X_val)

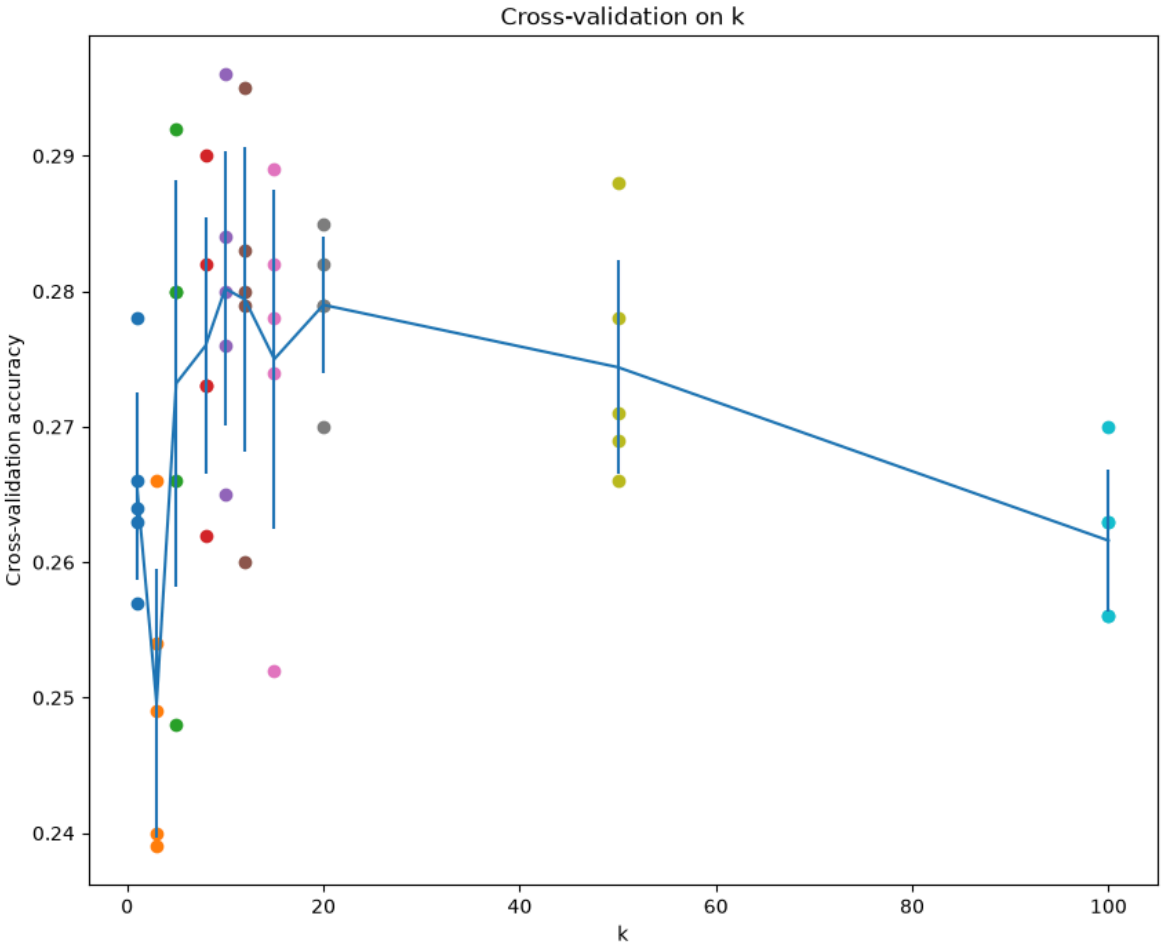
    for k in k_choices:
        y_val_pred = cv_classifier.predict_labels(val_dists, k=k)
        accuracy = np.mean(y_val_pred == y_val)
        k_to_accuracies.setdefault(k, []).append(accuracy)

    print("完成第 %d / %d 折" % (fold + 1, num_folds))
# Print out the computed accuracies
for k in sorted(k_to_accuracies):
    for accuracy in k_to_accuracies[k]:
        print('k = %d, accuracy = %f' % (k, accuracy))
```

```
分成了几份： 5 5
第一份图片形状： (1000, 3072)
第一份标签形状： (1000,)
完成第 1 / 5 折
完成第 2 / 5 折
完成第 3 / 5 折
完成第 4 / 5 折
完成第 5 / 5 折
k = 1, accuracy = 0.263000
k = 1, accuracy = 0.257000
k = 1, accuracy = 0.264000
k = 1, accuracy = 0.278000
k = 1, accuracy = 0.266000
k = 3, accuracy = 0.239000
k = 3, accuracy = 0.249000
k = 3, accuracy = 0.240000
k = 3, accuracy = 0.266000
k = 3, accuracy = 0.254000
k = 5, accuracy = 0.248000
k = 5, accuracy = 0.266000
k = 5, accuracy = 0.280000
k = 5, accuracy = 0.292000
k = 5, accuracy = 0.280000
k = 8, accuracy = 0.262000
k = 8, accuracy = 0.282000
k = 8, accuracy = 0.273000
k = 8, accuracy = 0.290000
k = 8, accuracy = 0.273000
k = 10, accuracy = 0.265000
k = 10, accuracy = 0.296000
k = 10, accuracy = 0.276000
k = 10, accuracy = 0.284000
k = 10, accuracy = 0.280000
k = 12, accuracy = 0.260000
k = 12, accuracy = 0.295000
k = 12, accuracy = 0.279000
k = 12, accuracy = 0.283000
k = 12, accuracy = 0.280000
k = 15, accuracy = 0.252000
k = 15, accuracy = 0.289000
k = 15, accuracy = 0.278000
k = 15, accuracy = 0.282000
k = 15, accuracy = 0.274000
k = 20, accuracy = 0.270000
k = 20, accuracy = 0.279000
k = 20, accuracy = 0.279000
k = 20, accuracy = 0.282000
k = 20, accuracy = 0.285000
k = 50, accuracy = 0.271000
k = 50, accuracy = 0.288000
k = 50, accuracy = 0.278000
k = 50, accuracy = 0.269000
k = 50, accuracy = 0.266000
k = 100, accuracy = 0.256000
k = 100, accuracy = 0.270000
k = 100, accuracy = 0.263000
k = 100, accuracy = 0.256000
k = 100, accuracy = 0.263000
```

In [27]:
```
# plot the raw observations
for k in k_choices:
    accuracies = k_to_accuracies[k]
    plt.scatter([k] * len(accuracies), accuracies)

# plot the trend line with error bars that correspond to standard deviation
accuracies_mean = np.array([np.mean(v) for k,v in sorted(k_to_accuracies.items())])
accuracies_std = np.array([np.std(v) for k,v in sorted(k_to_accuracies.items())])
plt.errorbar(k_choices, accuracies_mean, yerr=accuracies_std)
plt.title('Cross-validation on k')
plt.xlabel('k')
plt.ylabel('Cross-validation accuracy')
plt.show()
```



In [28]:
```
# Based on the cross-validation results above, choose the best value for k,
# retrain the classifier using all the training data, and test it on the test
# data. You should be able to get above 28% accuracy on the test data.
best_k = 10

classifier = KNearestNeighbor()
classifier.train(X_train, y_train)
y_test_pred = classifier.predict(X_test, k=best_k)

# Compute and display the accuracy
num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
```

```
Got 141 / 500 correct => accuracy: 0.282000
```

Inline Question 3

Which of the following statements about k-Nearest Neighbor (k-NN) are true in a classification setting, and for all k? Select all that apply.

1. The decision boundary of the k-NN classifier is linear.
2. The training error of a 1-NN will always be lower than or equal to that of 5-NN.
3. The test error of a 1-NN will always be lower than that of a 5-NN.
4. The time needed to classify a test example with the k-NN classifier grows with the size of the training set.
5. None of the above.

$\color{blue}{\textit Your Answer:2、4}$

$\color{blue}{\textit Your Explanation: 1．决策边界是线性的：错。 kNN 根据附近训练样本的标签分类，整体边界通常不规则，不保证是一条直线或一个超平面。 2．1-NN 的训练误差总是不高于 5-NN：对，按题目的通常约定。 在训练集上预测时，样本自身也在候选邻居中，到自己的距离为零，因此 1-NN 通常能正确预测每个训练样本，训练误差为零。5-NN 则可能被周围其他类别的样本投票影响。 这里默认没有“完全相同的输入却具有不同标签”的冲突样本，也没有排除样本自身。 3．1-NN 的测试误差总是低于 5-NN：错。 1-NN 容易受到噪声或偶然邻居影响；5-NN 的投票有时能改善结果，但也不保证更好。你刚才的实验中，5-NN 的准确率就略高于 1-NN。 4．分类耗时随训练集规模增长：对，针对本作业的实现。 预测一张图片时，需要与全部训练图片计算距离。训练图片越多，需要处理的距离也越多。 5．以上都不对：错。}$

In [ ]:

Generative AI Use: For the purposes of the assignments, the use of generative AI is subject to the same policies regarding collaboration. Just as with other collaborators, each student must write down the solutions independently of the output of the interaction and the submission should include a note denoting the nature of the collaboration. The use of generative AI tools to substantially complete sections of the assignments is not in line with the spirit of the assignments, and would be a violation of the Honor Code.

In [22]:
```
# This mounts your Google Drive to the Colab VM.
from google.colab import drive
drive.mount('/content/drive')

# TODO: Enter the foldername in your Drive where you have saved the unzipped
# assignment folder, e.g. 'cs231n/assignments/assignment1/'
FOLDERNAME = 'cs231n/assignments/assignment1/'
assert FOLDERNAME is not None, "[!] Enter the foldername."

# Now that we've mounted your Drive, this ensures that
# the Python interpreter of the Colab VM can load
# python files from within it.
import sys
sys.path.append('/content/drive/My Drive/{}'.format(FOLDERNAME))

# This downloads the CIFAR-10 dataset to your Drive
# if it doesn't already exist.
%cd /content/drive/My\ Drive/$FOLDERNAME/cs231n/datasets/
!bash get_datasets.sh
%cd /content/drive/My\ Drive/$FOLDERNAME
```

```
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
Cell In[22], line 2
      1 # This mounts your Google Drive to the Colab VM.
----> 2 from google.colab import drive
      3 drive.mount('/content/drive')
      4
      5 # TODO: Enter the foldername in your Drive where you have saved the unzip
ped

ModuleNotFoundError: No module named 'google'
```

Softmax Classifier exercise

Complete and hand in this completed worksheet (including its outputs and any supporting code outside of the worksheet) with your assignment submission. For more details see the assignments page on the course website.

In this exercise you will:

- implement a fully-vectorized **loss function** for the Softmax classifier.
- implement the fully-vectorized expression for its **analytic gradient**
- **check your implementation** using numerical gradient

- use a validation set to **tune the learning rate and regularization** strength
- **optimize** the loss function with **SGD**
- **visualize** the final learned weights

In [23]:
```
# Run some setup code for this notebook.
import random
import numpy as np
from cs231n.data_utils import load_CIFAR10
import matplotlib.pyplot as plt

# This is a bit of magic to make matplotlib figures appear inline in the
# notebook rather than in a new window.
%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# Some more magic so that the notebook will reload external python modules;
# see http://stackoverflow.com/questions/1907993/autoreload-of-modules-in-ipytho
%load_ext autoreload
%autoreload 2
```

```
The autoreload extension is already loaded. To reload it, use:
  %reload_ext autoreload
```

CIFAR-10 Data Loading and Preprocessing

In [4]:
```
# Load the raw CIFAR-10 data.
cifar10_dir = 'cs231n/datasets/cifar-10-batches-py'

# Cleaning up variables to prevent loading data multiple times (which may cause
try:
   del X_train, y_train
   del X_test, y_test
   print('Clear previously loaded data.')
except:
   pass

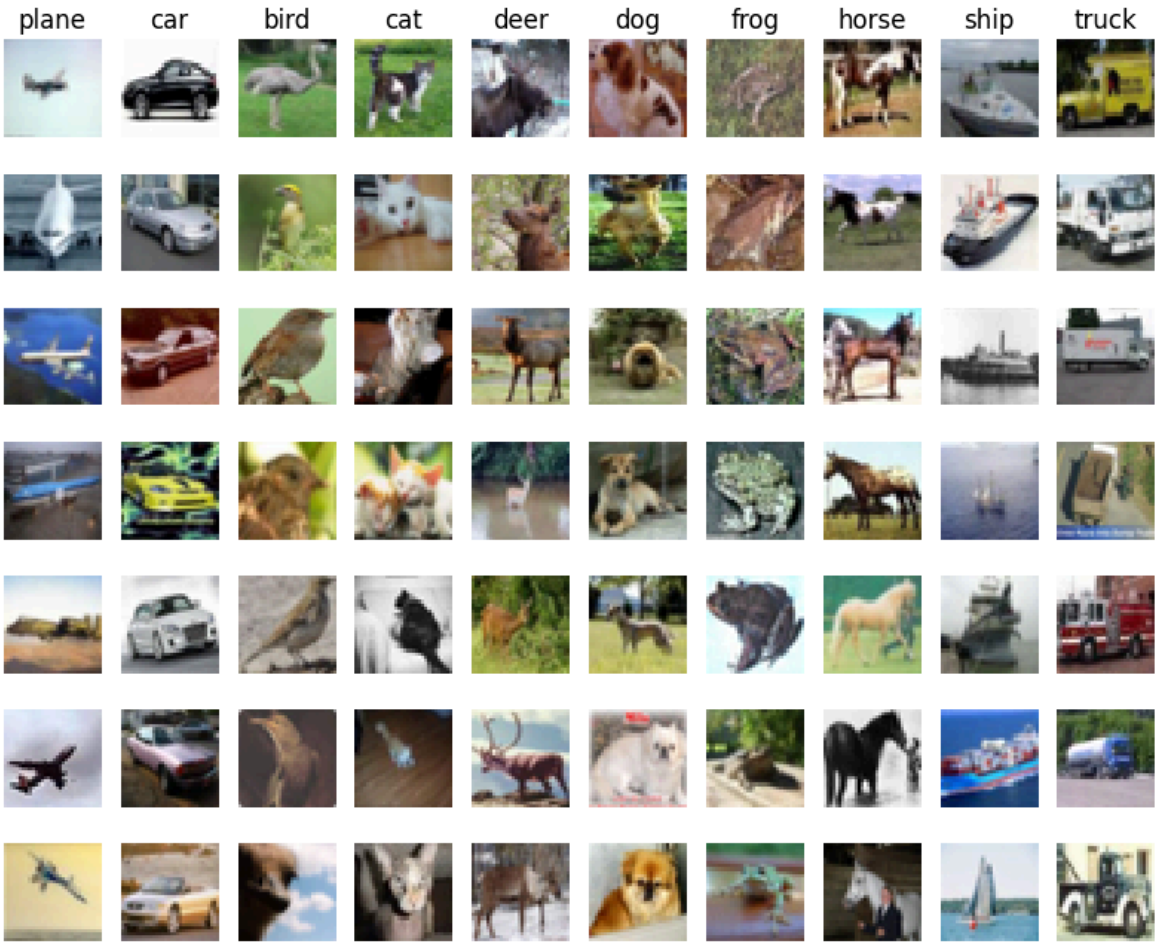
X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

# As a sanity check, we print out the size of the training and test data.
print('Training data shape: ', X_train.shape)
print('Training labels shape: ', y_train.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)
```

```
Training data shape:  (50000, 32, 32, 3)
Training labels shape:  (50000,)
Test data shape:  (10000, 32, 32, 3)
Test labels shape:  (10000,)
```

In [5]:
```
# Visualize some examples from the dataset.
# We show a few examples of training images from each class.
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship'
num_classes = len(classes)
samples_per_class = 7
for y, cls in enumerate(classes):
    idxs = np.flatnonzero(y_train == y)
```

```
    idxs = np.random.choice(idxs, samples_per_class, replace=False)
    for i, idx in enumerate(idxs):
        plt_idx = i * num_classes + y + 1
        plt.subplot(samples_per_class, num_classes, plt_idx)
        plt.imshow(X_train[idx].astype('uint8'))
        plt.axis('off')
        if i == 0:
            plt.title(cls)
plt.show()
```



In [6]:
```
# Split the data into train, val, and test sets. In addition we will
# create a small development set as a subset of the training data;
# we can use this for development so our code runs faster.
num_training = 49000
num_validation = 1000
num_test = 1000
num_dev = 500

# Our validation set will be num_validation points from the original
# training set.
mask = range(num_training, num_training + num_validation)
X_val = X_train[mask]
y_val = y_train[mask]

# Our training set will be the first num_train points from the original
# training set.
mask = range(num_training)
X_train = X_train[mask]
y_train = y_train[mask]

# We will also make a development set, which is a small subset of
```

```
# the training set.
mask = np.random.choice(num_training, num_dev, replace=False)
X_dev = X_train[mask]
y_dev = y_train[mask]

# We use the first num_test points of the original test set as our
# test set.
mask = range(num_test)
X_test = X_test[mask]
y_test = y_test[mask]

print('Train data shape: ', X_train.shape)
print('Train labels shape: ', y_train.shape)
print('Validation data shape: ', X_val.shape)
print('Validation labels shape: ', y_val.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)
```

```
Train data shape:  (49000, 32, 32, 3)
Train labels shape:  (49000,)
Validation data shape:  (1000, 32, 32, 3)
Validation labels shape:  (1000,)
Test data shape:  (1000, 32, 32, 3)
Test labels shape:  (1000,)
```

In [7]:
```
# Preprocessing: reshape the image data into rows
X_train = np.reshape(X_train, (X_train.shape[0], -1))
X_val = np.reshape(X_val, (X_val.shape[0], -1))
X_test = np.reshape(X_test, (X_test.shape[0], -1))
X_dev = np.reshape(X_dev, (X_dev.shape[0], -1))

# As a sanity check, print out the shapes of the data
print('Training data shape: ', X_train.shape)
print('Validation data shape: ', X_val.shape)
print('Test data shape: ', X_test.shape)
print('dev data shape: ', X_dev.shape)
```

```
Training data shape:  (49000, 3072)
Validation data shape:  (1000, 3072)
Test data shape:  (1000, 3072)
dev data shape:  (500, 3072)
```

In [8]:
```
# Preprocessing: subtract the mean image
# first: compute the image mean based on the training data
mean_image = np.mean(X_train, axis=0)
print(mean_image[:10]) # print a few of the elements
plt.figure(figsize=(4,4))
plt.imshow(mean_image.reshape((32,32,3)).astype('uint8')) # visualize the mean i
plt.show()

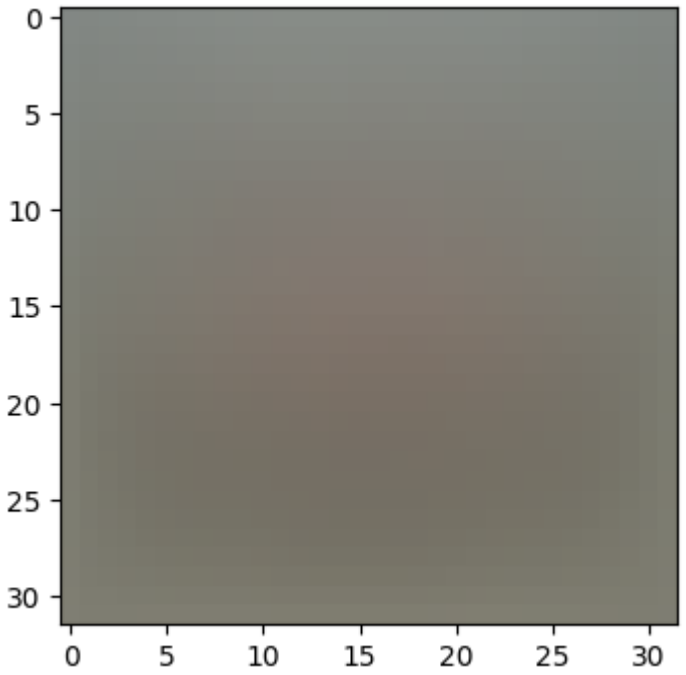
# second: subtract the mean image from train and test data
X_train -= mean_image
X_val -= mean_image
X_test -= mean_image
X_dev -= mean_image

# third: append the bias dimension of ones (i.e. bias trick) so that our classif
# only has to worry about optimizing a single weight matrix W.
X_train = np.hstack([X_train, np.ones((X_train.shape[0], 1))])
X_val = np.hstack([X_val, np.ones((X_val.shape[0], 1))])
X_test = np.hstack([X_test, np.ones((X_test.shape[0], 1))])
```

```
X_dev = np.hstack([X_dev, np.ones((X_dev.shape[0], 1))])

print(X_train.shape, X_val.shape, X_test.shape, X_dev.shape)
```

```
[130.64189796 135.98173469 132.47391837 130.05569388 135.34804082
 131.75402041 130.96055102 136.14328571 132.47636735 131.48467347]
```




```
(49000, 3073) (1000, 3073) (1000, 3073) (500, 3073)
```

Softmax Classifier

Your code for this section will all be written inside `cs231n/classifiers/softmax.py`.

As you can see, we have prefilled the function `softmax_loss_naive` which uses for loops to evaluate the softmax loss function.

In [24]:
```
# Evaluate the naive implementation of the loss we provided for you:
from cs231n.classifiers.softmax import softmax_loss_naive
import time

# generate a random Softmax classifier weight matrix of small numbers
W = np.random.randn(3073, 10) * 0.0001

loss, grad = softmax_loss_naive(W, X_dev, y_dev, 0.000005)
print('loss: %f' % (loss, ))

# As a rough sanity check, our loss should be something close to -log(0.1).
print('loss: %f' % loss)
print('sanity check: %f' % (-np.log(0.1)))
```

```
loss: 2.349093
loss: 2.349093
sanity check: 2.302585
```

Inline Question 1

Why do we expect our loss to be close to -log(0.1)? Explain briefly.**

$\color{blue}{\textit Your Answer:}$ 初始化时权重很小，各类别的得分接近，因此 Softmax 为每个类别分配的概率都约为 1/10，即 0.1。交叉熵损失是真实类别预测概率的负对数，所以初始损失应接近 $-\log(0.1) \approx 2.303$。

The `grad` returned from the function above is right now all zero. Derive and implement the gradient for the softmax loss function and implement it inline inside the function `softmax_loss_naive`. You will find it helpful to interleave your new code inside the existing function.

To check that you have correctly implemented the gradient, you can numerically estimate the gradient of the loss function and compare the numeric estimate to the gradient that you computed. We have provided code that does this for you:

In [25]:
```
# Once you've implemented the gradient, recompute it with the code below
# and gradient check it with the function we provided for you

# Compute the loss and its gradient at W.
loss, grad = softmax_loss_naive(W, X_dev, y_dev, 0.0)

# Numerically compute the gradient along several randomly chosen dimensions, and
# compare them with your analytically computed gradient. The numbers should matc
# almost exactly along all dimensions.
from cs231n.gradient_check import grad_check_sparse
f = lambda w: softmax_loss_naive(w, X_dev, y_dev, 0.0)[0]
grad_numerical = grad_check_sparse(f, W, grad)

# do the gradient check once again with regularization turned on
# you didn't forget the regularization gradient did you?
loss, grad = softmax_loss_naive(W, X_dev, y_dev, 5e1)
f = lambda w: softmax_loss_naive(w, X_dev, y_dev, 5e1)[0]
grad_numerical = grad_check_sparse(f, W, grad)
```

```
numerical: -4.538886 analytic: -4.538886, relative error: 2.148994e-09
numerical: -3.981820 analytic: -3.981820, relative error: 2.957404e-09
numerical: -2.010486 analytic: -2.010486, relative error: 3.541283e-08
numerical: -2.878273 analytic: -2.878273, relative error: 5.878980e-09
numerical: 1.349315 analytic: 1.349315, relative error: 2.893001e-08
numerical: -0.689596 analytic: -0.689596, relative error: 2.722776e-08
numerical: -2.498057 analytic: -2.498057, relative error: 9.957857e-09
numerical: 0.179440 analytic: 0.179440, relative error: 2.434461e-07
numerical: -2.326140 analytic: -2.326140, relative error: 3.301839e-09
numerical: 0.918419 analytic: 0.918419, relative error: 8.895683e-09
numerical: -3.464726 analytic: -3.464726, relative error: 6.579075e-09
numerical: 1.654322 analytic: 1.654322, relative error: 4.544990e-08
numerical: -4.613070 analytic: -4.613070, relative error: 1.386781e-10
numerical: 2.357071 analytic: 2.357071, relative error: 8.853985e-09
numerical: -1.011755 analytic: -1.011755, relative error: 1.535403e-08
numerical: 0.819733 analytic: 0.819733, relative error: 4.147805e-08
numerical: 0.011766 analytic: 0.011766, relative error: 1.452346e-06
numerical: 1.026311 analytic: 1.026311, relative error: 1.210915e-08
numerical: 0.383966 analytic: 0.383966, relative error: 8.487566e-08
numerical: -1.035221 analytic: -1.035221, relative error: 7.569856e-09
```

Inline Question 2

Although gradcheck is reliable softmax loss, it is possible that for SVM loss, once in a while, a dimension in the gradcheck will not match exactly. What could such a

discrepancy be caused by? Is it a reason for concern? What is a simple example in one dimension where a svm loss gradient check could fail? How would change the margin affect of the frequency of this happening?

Note that SVM loss for a sample (x_i, y_i) is defined as: $$L_i = \sum_{j\ne y_i}\max(0, s_j - s_{y_i} + \Delta)$$ where j iterates over all classes except the correct class y_i and s_j denotes the classifier score for j^{th} class. Δ is a scalar margin. For more information, refer to 'Multiclass Support Vector Machine loss' on this page.

Hint: the SVM loss function is not strictly speaking differentiable.

$\color{blue}{\textit Your Answer:}$ *SVM损失函数在正确类别与其他类别的分数差满足间隔条件时可能出现不可微点。梯度检查使用数值方法近似计算梯度，如果扰动前后的参数位于不可微点两侧，数值梯度就可能与解析梯度不一致。这种偶尔发生的情况通常不必担心，但如果频繁出现，就需要检查代码是否存在错误。

一维例子：设损失函数L(w)=max(0,w)，当w=0时，函数不可微。假设程序在该点将梯度设为0，采用中心差分计算数值梯度，结果为[L(h)-L(-h)]/(2h)=0.5，因此数值梯度与解析梯度不一致，梯度检查失败。

当margin增大时，损失函数的折点位置会发生变化。在常见情况下，较大的margin可能增加数值扰动跨越不可微点的机会，使梯度检查更容易失败。但实际发生频率还取决于数据、参数和数值步长。*

In [27]:
```
# Next implement the function softmax_loss_vectorized; for now only compute the
# we will implement the gradient in a moment.
tic = time.time()
loss_naive, grad_naive = softmax_loss_naive(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Naive loss: %e computed in %fs' % (loss_naive, toc - tic))

from cs231n.classifiers.softmax import softmax_loss_vectorized
tic = time.time()
loss_vectorized, _ = softmax_loss_vectorized(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Vectorized loss: %e computed in %fs' % (loss_vectorized, toc - tic))

# The losses should match but your vectorized implementation should be much fast
print('difference: %f' % (loss_naive - loss_vectorized))
```

```
Naive loss: 2.349093e+00 computed in 0.024482s
Vectorized loss: 2.349093e+00 computed in 0.003535s
difference: 0.000000
```

In [29]:
```
# Complete the implementation of softmax_loss_vectorized, and compute the gradie
# of the loss function in a vectorized way.

# The naive implementation and the vectorized implementation should match, but
# the vectorized version should still be much faster.
tic = time.time()
_, grad_naive = softmax_loss_naive(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Naive loss and gradient: computed in %fs' % (toc - tic))
```

```
tic = time.time()
_, grad_vectorized = softmax_loss_vectorized(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Vectorized loss and gradient: computed in %fs' % (toc - tic))

# The loss is a single number, so it is easy to compare the values computed
# by the two implementations. The gradient on the other hand is a matrix, so
# we use the Frobenius norm to compare them.
difference = np.linalg.norm(grad_naive - grad_vectorized, ord='fro')
print('difference: %f' % difference)
```

```
Naive loss and gradient: computed in 0.027411s
Vectorized loss and gradient: computed in 0.001890s
difference: 0.000000
```

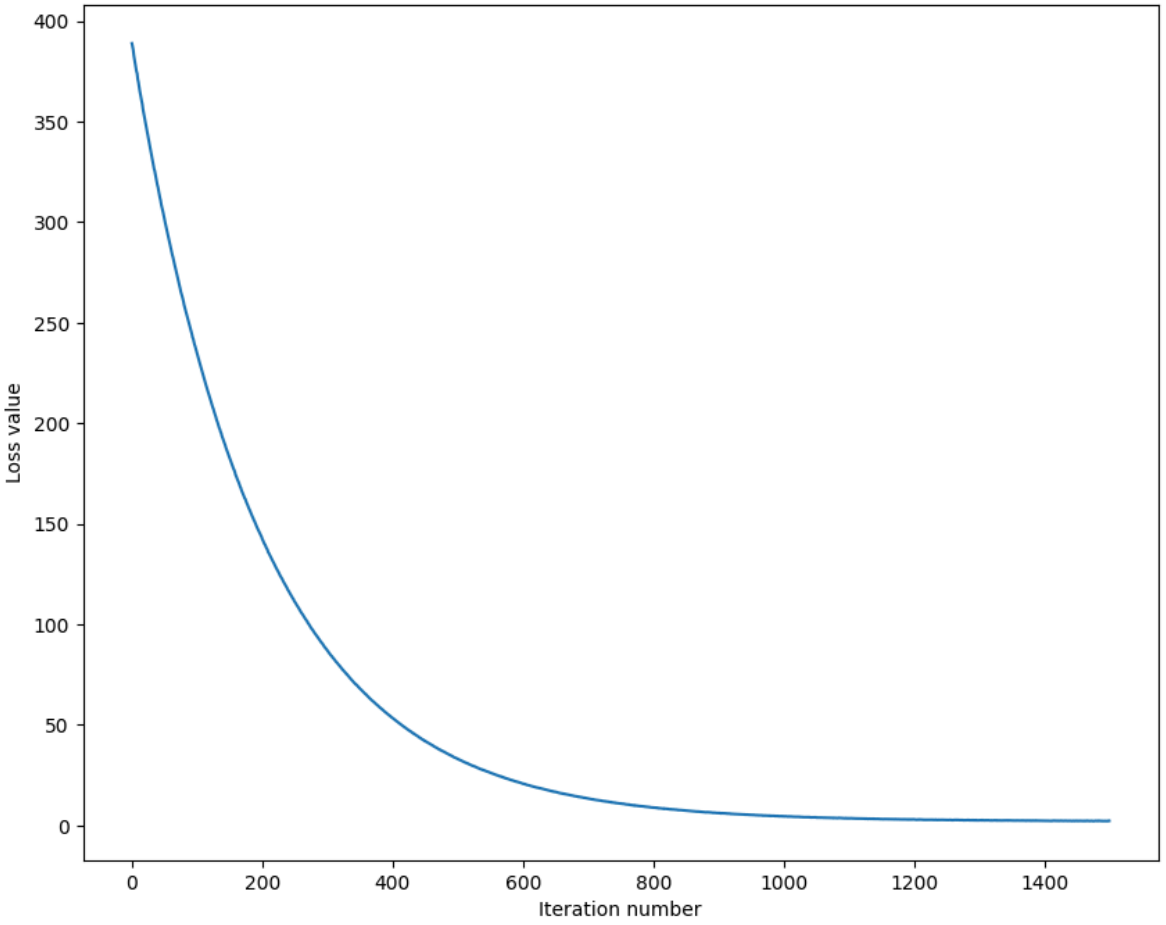
Stochastic Gradient Descent

We now have vectorized and efficient expressions for the loss, the gradient and our gradient matches the numerical gradient. We are therefore ready to do SGD to minimize the loss. Your code for this part will be written inside `cs231n/classifiers/linear_classifier.py`.

In [31]:
```
# In the file linear_classifier.py, implement SGD in the function
# LinearClassifier.train() and then run it with the code below.
from cs231n.classifiers import Softmax
softmax = Softmax()
tic = time.time()
loss_hist = softmax.train(X_train, y_train, learning_rate=1e-7, reg=2.5e4,
                          num_iters=1500, verbose=True)
toc = time.time()
print('That took %fs' % (toc - tic))
```

```
iteration 0 / 1500: loss 388.840458
iteration 100 / 1500: loss 234.404022
iteration 200 / 1500: loss 142.321143
iteration 300 / 1500: loss 86.858044
iteration 400 / 1500: loss 53.376462
iteration 500 / 1500: loss 32.989569
iteration 600 / 1500: loss 20.732691
iteration 700 / 1500: loss 13.362435
iteration 800 / 1500: loss 8.898150
iteration 900 / 1500: loss 6.180592
iteration 1000 / 1500: loss 4.622603
iteration 1100 / 1500: loss 3.534331
iteration 1200 / 1500: loss 3.007145
iteration 1300 / 1500: loss 2.559932
iteration 1400 / 1500: loss 2.320341
That took 3.965958s
```

In [32]:
```
# A useful debugging strategy is to plot the loss as a function of
# iteration number:
plt.plot(loss_hist)
plt.xlabel('Iteration number')
plt.ylabel('Loss value')
plt.show()
```



In [34]:
```
# Write the LinearClassifier.predict function and evaluate the performance on
# both the training and validation set
# You should get validation accuracy of about 0.34 (> 0.33).
y_train_pred = softmax.predict(X_train)
print('training accuracy: %f' % (np.mean(y_train == y_train_pred), ))
y_val_pred = softmax.predict(X_val)
print('validation accuracy: %f' % (np.mean(y_val == y_val_pred), ))
```

```
training accuracy: 0.350939
validation accuracy: 0.358000
```

In [35]:
```
# Save the trained model for autograder.
softmax.save("softmax.npy")
```

```
softmax.npy saved.
```

In [36]:
```
# Use the validation set to tune hyperparameters (regularization strength and
# learning rate). You should experiment with different ranges for the learning
# rates and regularization strengths; if you are careful you should be able to
# get a classification accuracy of about 0.365 (> 0.36) on the validation set.

# Note: you may see runtime/overflow warnings during hyper-parameter search.
# This may be caused by extreme values, and is not a bug.

# results is dictionary mapping tuples of the form
# (learning_rate, regularization_strength) to tuples of the form
# (training_accuracy, validation_accuracy). The accuracy is simply the fraction
# of data points that are correctly classified.
results = {}
best_val = -1   # The highest validation accuracy that we have seen so far.
best_softmax = None # The Softmax object that achieved the highest validation ra
```

```
################################################################################
# TODO:                                                                        #
# Write code that chooses the best hyperparameters by tuning on the validation #
# set. For each combination of hyperparameters, train a Softmax on the.        #
# training set, compute its accuracy on the training and validation sets, and  #
# store these numbers in the results dictionary. In addition, store the best   #
# validation accuracy in best_val and the Softmax object that achieves this.   #
# accuracy in best_softmax.                                                    #
#                                                                              #
# Hint: You should use a small value for num_iters as you develop your         #
# validation code so that the classifiers don't take much time to train; once  #
# you are confident that your validation code works, you should rerun the      #
# code with a larger value for num_iters.                                      #
################################################################################

# Provided as a reference. You may or may not want to change these hyperparamete
learning_rates = [1e-7, 1e-6]
regularization_strengths = [2.5e4, 1e4]

from cs231n.classifiers.linear_classifier import Softmax

for lr in learning_rates:
    for reg in regularization_strengths:
        np.random.seed(231)
        candidate = Softmax()

        candidate.train(
            X_train,
            y_train,
            learning_rate=lr,
            reg=reg,
            num_iters=2000,
            batch_size=200,
            verbose=False,
        )

        train_accuracy = np.mean(candidate.predict(X_train) == y_train)
        val_accuracy = np.mean(candidate.predict(X_val) == y_val)

        results[(lr, reg)] = (train_accuracy, val_accuracy)

        if val_accuracy > best_val:
            best_val = val_accuracy
            best_softmax = candidate

        print(
            f"完成: lr={lr:.1e}, reg={reg:.1e}, "
            f"验证准确率={val_accuracy:.2%}",
            flush=True,
        )

# Print out results.
for lr, reg in sorted(results):
    train_accuracy, val_accuracy = results[(lr, reg)]
    print('lr %e reg %e train accuracy: %f val accuracy: %f' % (
                lr, reg, train_accuracy, val_accuracy))

print('best validation accuracy achieved during cross-validation: %f' % best_val
```

```
完成：lr=1.0e-07, reg=2.5e+04, 验证准确率=36.20%
完成：lr=1.0e-07, reg=1.0e+04, 验证准确率=35.60%
完成：lr=1.0e-06, reg=2.5e+04, 验证准确率=35.50%
完成：lr=1.0e-06, reg=1.0e+04, 验证准确率=37.50%
lr 1.000000e-07 reg 1.000000e+04 train accuracy: 0.359816 val accuracy: 0.356000
lr 1.000000e-07 reg 2.500000e+04 train accuracy: 0.350245 val accuracy: 0.362000
lr 1.000000e-06 reg 1.000000e+04 train accuracy: 0.365347 val accuracy: 0.375000
lr 1.000000e-06 reg 2.500000e+04 train accuracy: 0.338184 val accuracy: 0.355000
best validation accuracy achieved during cross-validation: 0.375000
```

In [37]:

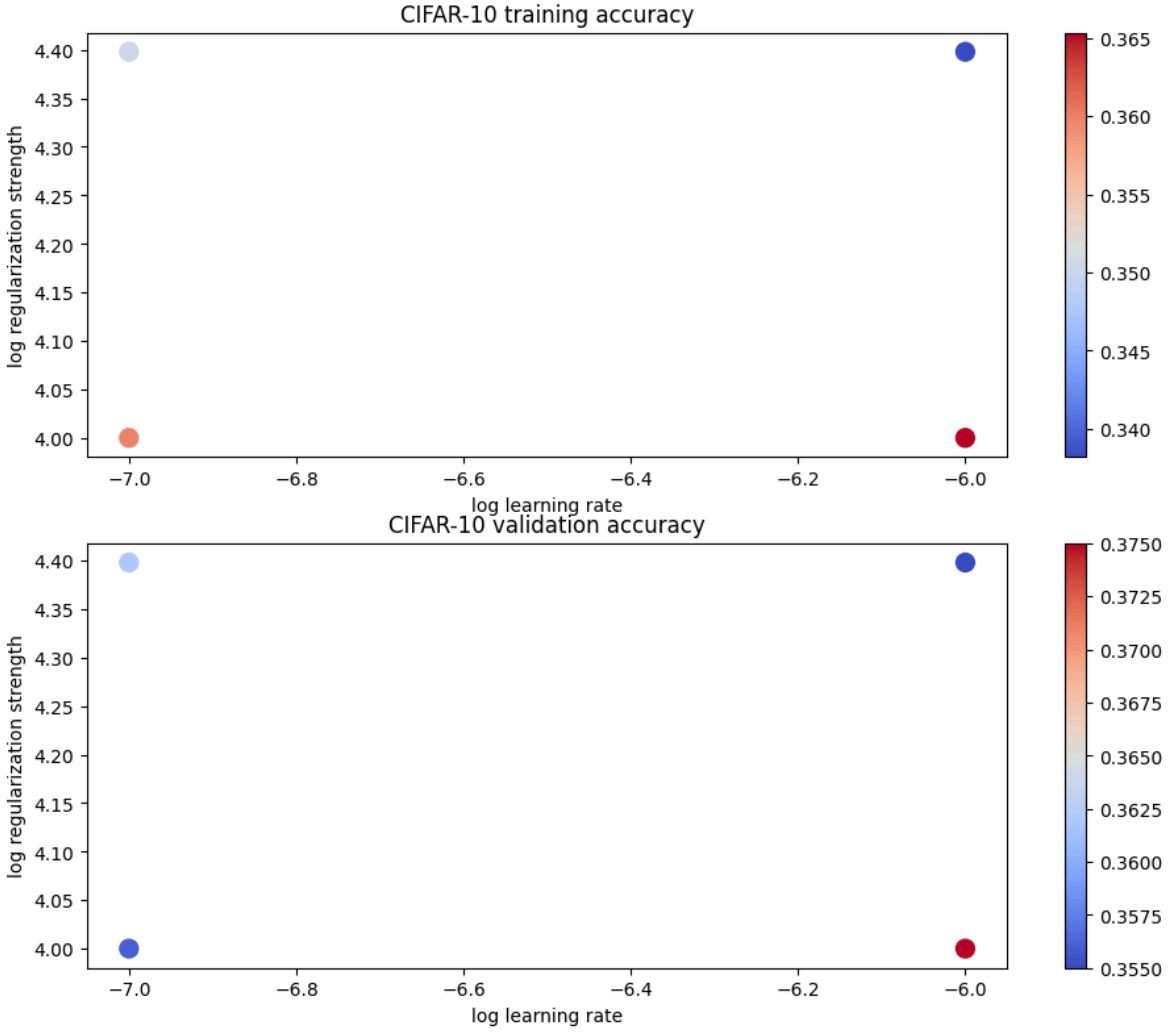
```python
# Visualize the cross-validation results
import math
import pdb

# pdb.set_trace()

x_scatter = [math.log10(x[0]) for x in results]
y_scatter = [math.log10(x[1]) for x in results]

# plot training accuracy
marker_size = 100
colors = [results[x][0] for x in results]
plt.subplot(2, 1, 1)
plt.tight_layout(pad=3)
plt.scatter(x_scatter, y_scatter, marker_size, c=colors, cmap=plt.cm.coolwarm)
plt.colorbar()
plt.xlabel('log learning rate')
plt.ylabel('log regularization strength')
plt.title('CIFAR-10 training accuracy')

# plot validation accuracy
colors = [results[x][1] for x in results] # default size of markers is 20
plt.subplot(2, 1, 2)
plt.scatter(x_scatter, y_scatter, marker_size, c=colors, cmap=plt.cm.coolwarm)
plt.colorbar()
plt.xlabel('log learning rate')
plt.ylabel('log regularization strength')
plt.title('CIFAR-10 validation accuracy')
plt.show()
```



In [39]:
```
# Evaluate the best softmax on test set
y_test_pred = best_softmax.predict(X_test)
test_accuracy = np.mean(y_test == y_test_pred)
print('Softmax classifier on raw pixels final test set accuracy: %f' % test_accu
```

```
Softmax classifier on raw pixels final test set accuracy: 0.366000
```

In [38]:
```
# Save best softmax model
best_softmax.save("best_softmax.npy")
```

```
best_softmax.npy saved.
```

In [40]:
```
# Visualize the learned weights for each class.
# Depending on your choice of learning rate and regularization strength, these m
# or may not be nice to look at.
w = best_softmax.W[:-1,:] # strip out the bias
w = w.reshape(32, 32, 3, 10)
w_min, w_max = np.min(w), np.max(w)
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship'
for i in range(10):
    plt.subplot(2, 5, i + 1)

    # Rescale the weights to be between 0 and 255
    wimg = 255.0 * (w[:, :, :, i].squeeze() - w_min) / (w_max - w_min)
    plt.imshow(wimg.astype('uint8'))
    plt.axis('off')
    plt.title(classes[i])
```



Inline question 3

Describe what your visualized Softmax classifier weights look like, and offer a brief explanation for why they look the way they do.

$\color{blue}{\textit Your Answer:}$ Softmax 分类器的权重图通常像各类别的模糊模板，呈现粗略的颜色分布和轮廓，而不是清晰的物体图像。这是因为线性分类器通过各位置像素的加权求和进行分类，同一类别中物体的位置、姿态和背景差异会使模板变得模糊。权重反映的是支持或抑制该类别得分的像素特征，并非简单的平均图像。

Inline Question 4 - *True or False*

Suppose the overall training loss is defined as the sum of the per-datapoint loss over all training examples. It is possible to add a new datapoint to a training set that would change the softmax loss, but leave the SVM loss unchanged.

$\color{blue}{\textit Your Answer:}$True

$\color{blue}{\textit Your Explanation:}$在模型参数固定时，如果新增样本的正确类别得分比所有错误类别得分至少高出规定间隔，那么它的 SVM 损失为零，总损失不变。而对于有限的类别得分，Softmax 给正确类别的概率仍小于 1，因此该样本的交叉熵损失大于零，会增加题目所定义的损失总和。

In [ ]:

Generative AI Use: For the purposes of the assignments, the use of generative AI is subject to the same policies regarding collaboration. Just as with other collaborators, each student must write down the solutions independently of the output of the interaction and the submission should include a note denoting the nature of the collaboration. The use of generative AI tools to substantially complete sections of the assignments is not in line with the spirit of the assignments, and would be a violation of the Honor Code.

In [1]:
```
from pathlib import Path
import sys

# 切换到作业目录
%cd E:/CS231n/assignment1

# 确保能导入 cs231n
project_dir = Path.cwd()
if str(project_dir) not in sys.path:
    sys.path.insert(0, str(project_dir))

# 检查数据集
data_dir = project_dir / "cs231n" / "datasets" / "cifar-10-batches-py"

print("当前目录：", project_dir)
print("cs231n 是否存在：", (project_dir / "cs231n").is_dir())
print("数据集是否存在：", (data_dir / "data_batch_1").is_file())
```

```
E:\CS231n\assignment1
当前目录： E:\CS231n\assignment1
cs231n 是否存在： True
数据集是否存在： True
```

Fully-Connected Neural Nets

In this exercise we will implement fully-connected networks using a modular approach. For each layer we will implement a `forward` and a `backward` function. The `forward` function will receive inputs, weights, and other parameters and will return both an output and a `cache` object storing data needed for the backward pass, like this:

```
def layer_forward(x, w):
  """ Receive inputs x and weights w """
  # Do some computations ...
  z = # ... some intermediate value
  # Do some more computations ...
  out = # the output

  cache = (x, w, z, out) # Values we need to compute gradients

  return out, cache
```

The backward pass will receive upstream derivatives and the `cache` object, and will return gradients with respect to the inputs and weights, like this:

```
def layer_backward(dout, cache):
  """
  Receive dout (derivative of loss with respect to outputs) and cache,
  and compute derivative with respect to inputs.
  """
  # Unpack cache values
  x, w, z, out = cache

  # Use values in cache to compute derivatives
  dx = # Derivative of loss with respect to x
  dw = # Derivative of loss with respect to w

  return dx, dw
```

After implementing a bunch of layers this way, we will be able to easily combine them to build classifiers with different architectures.

In [2]:
```
# As usual, a bit of setup
from __future__ import print_function
import time
import numpy as np
import matplotlib.pyplot as plt
from cs231n.classifiers.fc_net import *
from cs231n.data_utils import get_CIFAR10_data
from cs231n.gradient_check import eval_numerical_gradient, eval_numerical_gradie
from cs231n.solver import Solver

%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# for auto-reloading external modules
# see http://stackoverflow.com/questions/1907993/autoreload-of-modules-in-ipytho
%load_ext autoreload
%autoreload 2

def rel_error(x, y):
  """ returns relative error """
  return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y))))
```

In [3]:
```
# Load the (preprocessed) CIFAR10 data.

data = get_CIFAR10_data()
for k, v in list(data.items()):
  print(('%s: ' % k, v.shape))
```

```
('X_train: ', (49000, 3, 32, 32))
('y_train: ', (49000,))
('X_val: ', (1000, 3, 32, 32))
('y_val: ', (1000,))
('X_test: ', (1000, 3, 32, 32))
('y_test: ', (1000,))
```

Affine layer: forward

Open the file `cs231n/layers.py` and implement the `affine_forward` function.

Once you are done you can test your implementaion by running the following:

In [4]:
```
# Test the affine_forward function

num_inputs = 2
input_shape = (4, 5, 6)
output_dim = 3

input_size = num_inputs * np.prod(input_shape)
weight_size = output_dim * np.prod(input_shape)

x = np.linspace(-0.1, 0.5, num=input_size).reshape(num_inputs, *input_shape)
w = np.linspace(-0.2, 0.3, num=weight_size).reshape(np.prod(input_shape), output
b = np.linspace(-0.3, 0.1, num=output_dim)

out, _ = affine_forward(x, w, b)
correct_out = np.array([[ 1.49834967,  1.70660132,  1.91485297],
                        [ 3.25553199,  3.5141327,   3.77273342]])

# Compare your output with ours. The error should be around e-9 or less.
print('Testing affine_forward function:')
print('difference: ', rel_error(out, correct_out))
```

```
Testing affine_forward function:
difference:  9.769849468192957e-10
```

Affine layer: backward

Now implement the `affine_backward` function and test your implementation using numeric gradient checking.

In [5]:
```
# Test the affine_backward function
np.random.seed(231)
x = np.random.randn(10, 2, 3)
w = np.random.randn(6, 5)
b = np.random.randn(5)
dout = np.random.randn(10, 5)

dx_num = eval_numerical_gradient_array(lambda x: affine_forward(x, w, b)[0], x,
dw_num = eval_numerical_gradient_array(lambda w: affine_forward(x, w, b)[0], w,
db_num = eval_numerical_gradient_array(lambda b: affine_forward(x, w, b)[0], b,

_, cache = affine_forward(x, w, b)
dx, dw, db = affine_backward(dout, cache)

# The error should be around e-10 or less
print('Testing affine_backward function:')
print('dx error: ', rel_error(dx_num, dx))
print('dw error: ', rel_error(dw_num, dw))
print('db error: ', rel_error(db_num, db))
```

```
Testing affine_backward function:
dx error:  5.399100368651805e-11
dw error:  9.904211865398145e-11
db error:  2.4122867568119087e-11
```

ReLU activation: forward

Implement the forward pass for the ReLU activation function in the `relu_forward` function and test your implementation using the following:

In [6]:
```
# Test the relu_forward function

x = np.linspace(-0.5, 0.5, num=12).reshape(3, 4)

out, _ = relu_forward(x)
correct_out = np.array([[ 0.,          0.,          0.,          0.,        ],
                        [ 0.,          0.,          0.04545455,  0.13636364,],
                        [ 0.22727273,  0.31818182,  0.40909091,  0.5,       ]])

# Compare your output with ours. The error should be on the order of e-8
print('Testing relu_forward function:')
print('difference: ', rel_error(out, correct_out))
```

```
Testing relu_forward function:
difference:  4.999999798022158e-08
```

ReLU activation: backward

Now implement the backward pass for the ReLU activation function in the `relu_backward` function and test your implementation using numeric gradient checking:

In [7]:
```
np.random.seed(231)
x = np.random.randn(10, 10)
dout = np.random.randn(*x.shape)

dx_num = eval_numerical_gradient_array(lambda x: relu_forward(x)[0], x, dout)

_, cache = relu_forward(x)
dx = relu_backward(dout, cache)

# The error should be on the order of e-12
print('Testing relu_backward function:')
print('dx error: ', rel_error(dx_num, dx))
```

```
Testing relu_backward function:
dx error:  3.2756349136310288e-12
```

Inline Question 1:

We've only asked you to implement ReLU, but there are a number of different activation functions that one could use in neural networks, each with its pros and cons. In particular, an issue commonly seen with activation functions is getting zero (or close to zero) gradient flow during backpropagation. Which of the following activation functions have this problem? If you consider these functions in the one dimensional case, what types of input would lead to this behaviour?

1. Sigmoid
2. ReLU
3. Leaky ReLU

$\color{blue}{\textit Your Answer:}$ Sigmoid： 当输入是很大的正数或负数时，输出接近 1 或 0，进入饱和区。其导数为 σ(x)(1−σ(x))，此时接近零，容易导致梯度消失。

ReLU： 当输入小于 0 时，输出和导数都为零，会阻断梯度传播；当输入大于 0 时，导数为 1。输入等于 0 时不可导，实现中通常将导数设为 0。

Leaky ReLU： 当输入小于 0 时，导数为一个较小的正常数 α；输入大于 0 时，导数为 1。因此它避免了 ReLU 在负半轴上梯度完全为零的问题，但如果 α 很小，负半轴上的梯度仍可能接近零。

"Sandwich" layers

There are some common patterns of layers that are frequently used in neural nets. For example, affine layers are frequently followed by a ReLU nonlinearity. To make these common patterns easy, we define several convenience layers in the file `cs231n/layer_utils.py`.

For now take a look at the `affine_relu_forward` and `affine_relu_backward` functions, and run the following to numerically gradient check the backward pass:

In [8]:
```
from cs231n.layer_utils import affine_relu_forward, affine_relu_backward
np.random.seed(231)
x = np.random.randn(2, 3, 4)
w = np.random.randn(12, 10)
b = np.random.randn(10)
dout = np.random.randn(2, 10)

out, cache = affine_relu_forward(x, w, b)
dx, dw, db = affine_relu_backward(dout, cache)

dx_num = eval_numerical_gradient_array(lambda x: affine_relu_forward(x, w, b)[0]
dw_num = eval_numerical_gradient_array(lambda w: affine_relu_forward(x, w, b)[0]
db_num = eval_numerical_gradient_array(lambda b: affine_relu_forward(x, w, b)[0]

# Relative error should be around e-10 or less
print('Testing affine_relu_forward and affine_relu_backward:')
print('dx error: ', rel_error(dx_num, dx))
print('dw error: ', rel_error(dw_num, dw))
print('db error: ', rel_error(db_num, db))
```

```
Testing affine_relu_forward and affine_relu_backward:
dx error:  2.299579177309368e-11
dw error:  8.162011105764925e-11
db error:  7.826724021458994e-12
```

Loss layers: Softmax

Now implement the loss and gradient for softmax in the `softmax_loss` function in `cs231n/layers.py`. These should be similar to what you implemented in `cs231n/classifiers/softmax.py`. Other loss functions (e.g. `svm_loss`) can also be implemented in a modular way, however, it is not required for this assignment.

You can make sure that the implementations are correct by running the following:

In [9]:
```
np.random.seed(231)
num_classes, num_inputs = 10, 50
x = 0.001 * np.random.randn(num_inputs, num_classes)
y = np.random.randint(num_classes, size=num_inputs)


dx_num = eval_numerical_gradient(lambda x: softmax_loss(x, y)[0], x, verbose=Fal
loss, dx = softmax_loss(x, y)

# Test softmax_loss function. Loss should be close to 2.3 and dx error should be
print('\nTesting softmax_loss:')
print('loss: ', loss)
print('dx error: ', rel_error(dx_num, dx))
```

```
Testing softmax_loss:
loss:  2.302545844500738
dx error:  9.384673161989355e-09
```

Two-layer network

Open the file `cs231n/classifiers/fc_net.py` and complete the implementation of the `TwoLayerNet` class. Read through it to make sure you understand the API. You can run the cell below to test your implementation.

In [10]:
```
np.random.seed(231)
N, D, H, C = 3, 5, 50, 7
X = np.random.randn(N, D)
y = np.random.randint(C, size=N)

std = 1e-3
model = TwoLayerNet(input_dim=D, hidden_dim=H, num_classes=C, weight_scale=std)

print('Testing initialization ... ')
W1_std = abs(model.params['W1'].std() - std)
b1 = model.params['b1']
W2_std = abs(model.params['W2'].std() - std)
b2 = model.params['b2']
assert W1_std < std / 10, 'First layer weights do not seem right'
assert np.all(b1 == 0), 'First layer biases do not seem right'
assert W2_std < std / 10, 'Second layer weights do not seem right'
assert np.all(b2 == 0), 'Second layer biases do not seem right'

print('Testing test-time forward pass ... ')
model.params['W1'] = np.linspace(-0.7, 0.3, num=D*H).reshape(D, H)
model.params['b1'] = np.linspace(-0.1, 0.9, num=H)
model.params['W2'] = np.linspace(-0.3, 0.4, num=H*C).reshape(H, C)
model.params['b2'] = np.linspace(-0.9, 0.1, num=C)
X = np.linspace(-5.5, 4.5, num=N*D).reshape(D, N).T
scores = model.loss(X)
```

```
correct_scores = np.asarray(
  [[11.53165108,  12.2917344,   13.05181771,  13.81190102,  14.57198434, 15.3320
   [12.05769098,  12.74614105,  13.43459113,  14.1230412,   14.81149128, 15.4999
   [12.58373087,  13.20054771,  13.81736455,  14.43418138,  15.05099822, 15.6678
scores_diff = np.abs(scores - correct_scores).sum()
assert scores_diff < 1e-6, 'Problem with test-time forward pass'

print('Testing training loss (no regularization)')
y = np.asarray([0, 5, 1])
loss, grads = model.loss(X, y)
correct_loss = 3.4702243556
assert abs(loss - correct_loss) < 1e-10, 'Problem with training-time loss'

model.reg = 1.0
loss, grads = model.loss(X, y)
correct_loss = 26.5948426952
assert abs(loss - correct_loss) < 1e-10, 'Problem with regularization loss'

# Errors should be around e-7 or less
for reg in [0.0, 0.7]:
  print('Running numeric gradient check with reg = ', reg)
  model.reg = reg
  loss, grads = model.loss(X, y)

  for name in sorted(grads):
    f = lambda _: model.loss(X, y)[0]
    grad_num = eval_numerical_gradient(f, model.params[name], verbose=False)
    print('%s relative error: %.2e' % (name, rel_error(grad_num, grads[name])))
```

```
Testing initialization ...
Testing test-time forward pass ...
Testing training loss (no regularization)
Running numeric gradient check with reg =  0.0
W1 relative error: 1.83e-08
W2 relative error: 3.12e-10
b1 relative error: 9.83e-09
b2 relative error: 4.33e-10
Running numeric gradient check with reg =  0.7
W1 relative error: 2.53e-07
W2 relative error: 2.85e-08
b1 relative error: 1.56e-08
b2 relative error: 7.76e-10
```

Solver

Open the file `cs231n/solver.py` and read through it to familiarize yourself with the API. After doing so, use a `Solver` instance to train a `TwoLayerNet` that achieves about `36%` accuracy on the validation set.

In [14]:
```
input_size = 32 * 32 * 3
hidden_size = 50
num_classes = 10
model = TwoLayerNet(input_size, hidden_size, num_classes)
solver = None

################################################################################
# TODO: Use a Solver instance to train a TwoLayerNet that achieves about 36% #
```

```
# accuracy on the validation set.                                              #
################################################################################
solver = Solver(
    model,
    data,
    update_rule='sgd',
    optim_config={
        'learning_rate': 1e-3,
    },
    lr_decay=0.95,
    num_epochs=10,
    batch_size=100,
    print_every=100,
    verbose=True,
)

solver.train()

print(f"最佳验证集准确率：{solver.best_val_acc:.2%}")
################################################################################
#                              END OF YOUR CODE                                #
################################################################################
```

```
(Iteration 1 / 4900) loss: 2.298369
(Epoch 0 / 10) train acc: 0.122000; val_acc: 0.141000
(Iteration 101 / 4900) loss: 1.840313
(Iteration 201 / 4900) loss: 1.726657
(Iteration 301 / 4900) loss: 1.552888
(Iteration 401 / 4900) loss: 1.555883
(Epoch 1 / 10) train acc: 0.438000; val_acc: 0.409000
(Iteration 501 / 4900) loss: 1.651887
(Iteration 601 / 4900) loss: 1.415009
(Iteration 701 / 4900) loss: 1.484950
(Iteration 801 / 4900) loss: 1.384124
(Iteration 901 / 4900) loss: 1.429565
(Epoch 2 / 10) train acc: 0.480000; val_acc: 0.451000
(Iteration 1001 / 4900) loss: 1.424853
(Iteration 1101 / 4900) loss: 1.548256
(Iteration 1201 / 4900) loss: 1.495457
(Iteration 1301 / 4900) loss: 1.640258
(Iteration 1401 / 4900) loss: 1.257257
(Epoch 3 / 10) train acc: 0.516000; val_acc: 0.468000
(Iteration 1501 / 4900) loss: 1.450194
(Iteration 1601 / 4900) loss: 1.475270
(Iteration 1701 / 4900) loss: 1.374360
(Iteration 1801 / 4900) loss: 1.349628
(Iteration 1901 / 4900) loss: 1.554601
(Epoch 4 / 10) train acc: 0.531000; val_acc: 0.481000
(Iteration 2001 / 4900) loss: 1.537880
(Iteration 2101 / 4900) loss: 1.468909
(Iteration 2201 / 4900) loss: 1.425164
(Iteration 2301 / 4900) loss: 1.273663
(Iteration 2401 / 4900) loss: 1.374983
(Epoch 5 / 10) train acc: 0.513000; val_acc: 0.478000
(Iteration 2501 / 4900) loss: 1.341371
(Iteration 2601 / 4900) loss: 1.380097
(Iteration 2701 / 4900) loss: 1.316468
(Iteration 2801 / 4900) loss: 1.218807
(Iteration 2901 / 4900) loss: 1.147582
(Epoch 6 / 10) train acc: 0.547000; val_acc: 0.486000
(Iteration 3001 / 4900) loss: 1.398115
(Iteration 3101 / 4900) loss: 1.182891
(Iteration 3201 / 4900) loss: 1.358050
(Iteration 3301 / 4900) loss: 1.373527
(Iteration 3401 / 4900) loss: 1.339221
(Epoch 7 / 10) train acc: 0.561000; val_acc: 0.478000
(Iteration 3501 / 4900) loss: 1.354092
(Iteration 3601 / 4900) loss: 1.523348
(Iteration 3701 / 4900) loss: 1.285263
(Iteration 3801 / 4900) loss: 1.394352
(Iteration 3901 / 4900) loss: 1.153504
(Epoch 8 / 10) train acc: 0.566000; val_acc: 0.511000
(Iteration 4001 / 4900) loss: 1.289403
(Iteration 4101 / 4900) loss: 1.311952
(Iteration 4201 / 4900) loss: 1.099059
(Iteration 4301 / 4900) loss: 1.205874
(Iteration 4401 / 4900) loss: 1.347339
(Epoch 9 / 10) train acc: 0.561000; val_acc: 0.485000
(Iteration 4501 / 4900) loss: 1.254664
(Iteration 4601 / 4900) loss: 1.320722
(Iteration 4701 / 4900) loss: 1.503059
(Iteration 4801 / 4900) loss: 1.068641
```

```
(Epoch 10 / 10) train acc: 0.563000; val_acc: 0.503000
最佳验证集准确率：51.10%
```

Debug the training

With the default parameters we provided above, you should get a validation accuracy of about 0.36 on the validation set. This isn't very good.

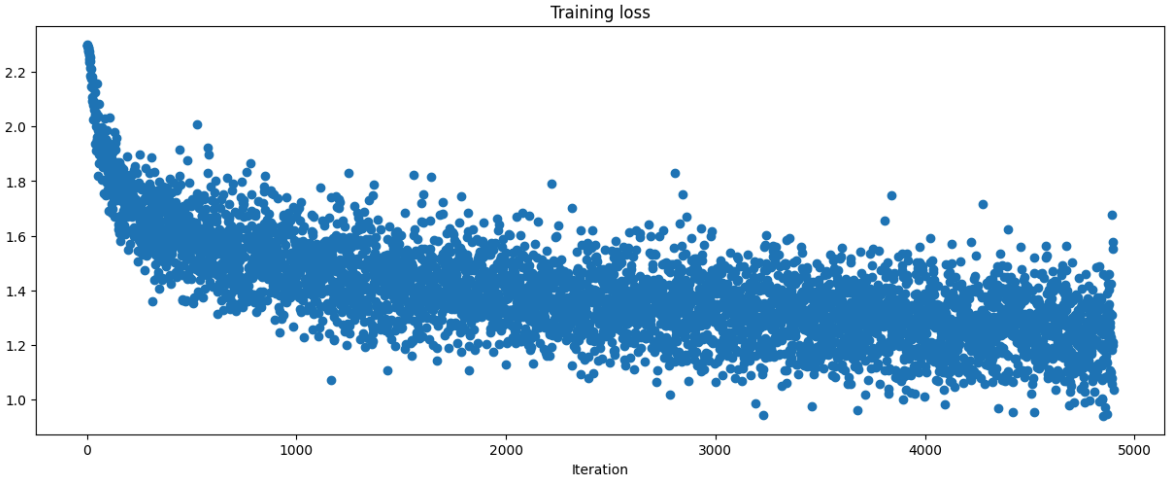
One strategy for getting insight into what's wrong is to plot the loss function and the accuracies on the training and validation sets during optimization.

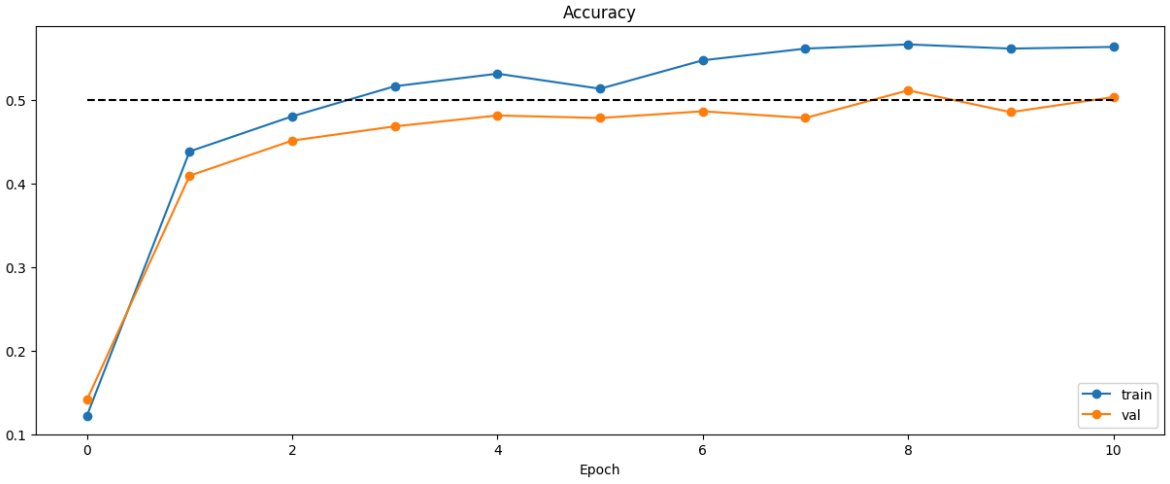
Another strategy is to visualize the weights that were learned in the first layer of the network. In most neural networks trained on visual data, the first layer weights typically show some visible structure when visualized.

In [15]:
```
# Run this cell to visualize training loss and train / val accuracy

plt.subplot(2, 1, 1)
plt.title('Training loss')
plt.plot(solver.loss_history, 'o')
plt.xlabel('Iteration')

plt.subplot(2, 1, 2)
plt.title('Accuracy')
plt.plot(solver.train_acc_history, '-o', label='train')
plt.plot(solver.val_acc_history, '-o', label='val')
plt.plot([0.5] * len(solver.val_acc_history), 'k--')
plt.xlabel('Epoch')
plt.legend(loc='lower right')
plt.gcf().set_size_inches(15, 12)
plt.show()
```





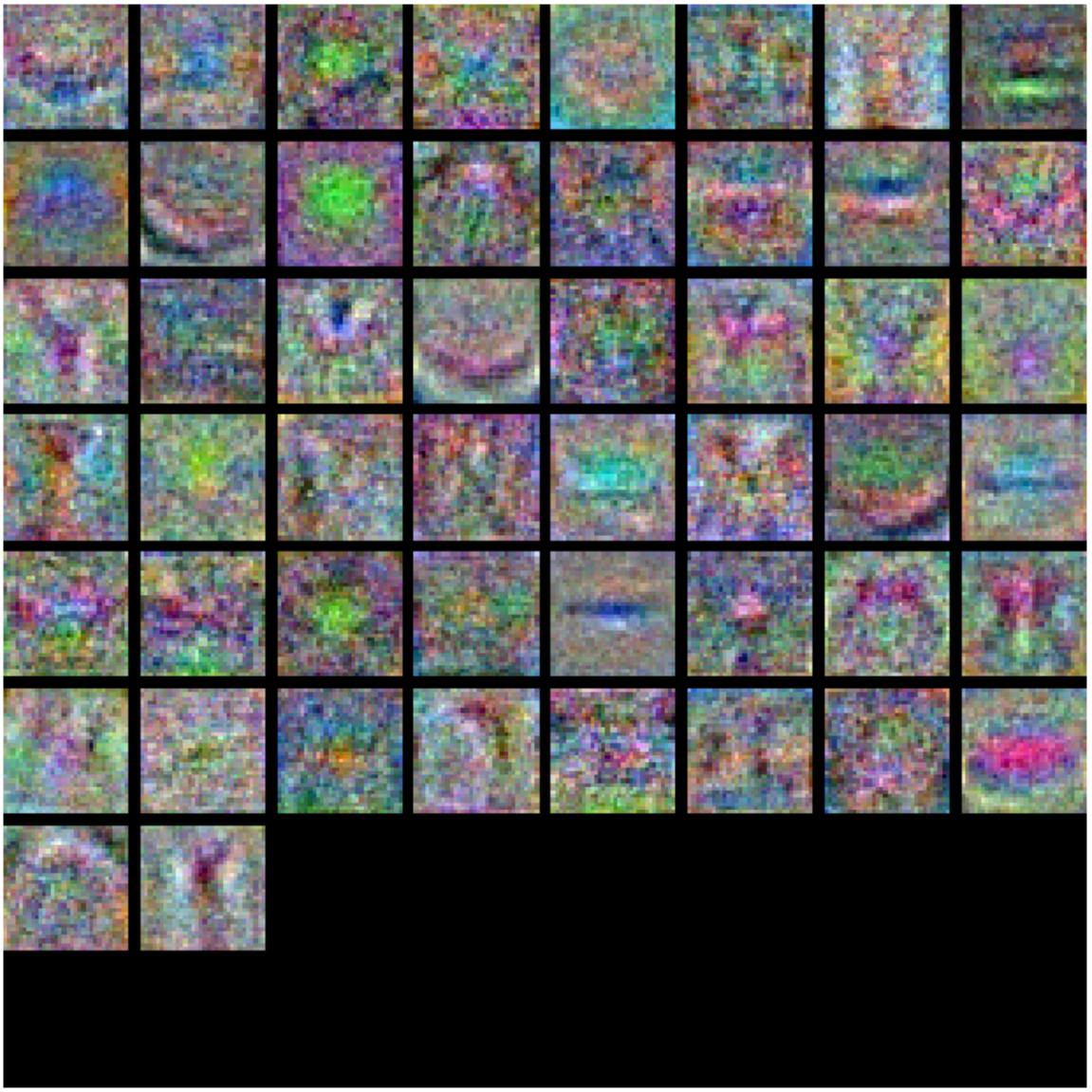
In [16]:

```
from cs231n.vis_utils import visualize_grid

# Visualize the weights of the network

def show_net_weights(net):
    W1 = net.params['W1']
    W1 = W1.reshape(3, 32, 32, -1).transpose(3, 1, 2, 0)
    plt.imshow(visualize_grid(W1, padding=3).astype('uint8'))
    plt.gca().axis('off')
    plt.show()

show_net_weights(model)
```



Tune your hyperparameters

What's wrong?. Looking at the visualizations above, we see that the loss is decreasing more or less linearly, which seems to suggest that the learning rate may be too low. Moreover, there is no gap between the training and validation accuracy, suggesting that the model we used has low capacity, and that we should increase its size. On the other hand, with a very large model we would expect to see more overfitting, which would manifest itself as a very large gap between the training and validation accuracy.

Tuning. Tuning the hyperparameters and developing intuition for how they affect the final performance is a large part of using Neural Networks, so we want you to get a lot of practice. Below, you should experiment with different values of the various hyperparameters, including hidden layer size, learning rate, numer of training epochs, and regularization strength. You might also consider tuning the learning rate decay, but you should be able to get good performance using the default value.

Approximate results. You should be aim to achieve a classification accuracy of greater than 48% on the validation set. Our best network gets over 52% on the validation set.

Experiment: You goal in this exercise is to get as good of a result on CIFAR-10 as you can (52% could serve as a reference), with a fully-connected Neural Network. Feel free implement your own techniques (e.g. PCA to reduce dimensionality, or adding dropout, or adding features to the solver, etc.).

In [11]:
```
best_model = None



################################################################################
# TODO: Tune hyperparameters using the validation set. Store your best trained
# model in best_model.
#
# To help debug your network, it may help to use visualizations similar to the
# ones we used above; these visualizations will have significant qualitative
# differences from the ones we saw above for the poorly tuned network.
#
# Tweaking hyperparameters by hand can be fun, but you might find it useful to
# write code to sweep through possible combinations of hyperparameters
# automatically like we did on thexs previous exercises.
################################################################################
best_val_acc = -1
results = {}

learning_rates = [5e-4, 1e-3]
regularization_strengths = [0.0, 0.1]
hidden_sizes = [100, 200]

for hidden_size in hidden_sizes:
    for lr in learning_rates:
        for reg in regularization_strengths:
            # 固定随机种子，方便比较和复现
            np.random.seed(231)

            candidate = TwoLayerNet(
                input_dim=32 * 32 * 3,
                hidden_dim=hidden_size,
                num_classes=10,
                weight_scale=1e-3,
                reg=reg,
            )

            candidate_solver = Solver(
                candidate,
                data,
                update_rule='sgd',
                optim_config={'learning_rate': lr},
                lr_decay=0.95,
                num_epochs=10,
                batch_size=100,
                verbose=False,
            )

            candidate_solver.train()
```

```
            val_acc = candidate_solver.best_val_acc

            results[(hidden_size, lr, reg)] = val_acc
            print(
                f"隐藏单元：{hidden_size}，学习率：{lr:.1e}，"
                f"正则化：{reg:.1e}，验证准确率：{val_acc:.2%}",
                flush=True,
            )

            if val_acc > best_val_acc:
                best_val_acc = val_acc
                best_model = candidate

print(f"\n最佳验证准确率：{best_val_acc:.2%}")
################################################################################
#                              END OF YOUR CODE                                #
################################################################################
```

```
隐藏单元：100，学习率：5.0e-04，正则化：0.0e+00，验证准确率：53.20%
隐藏单元：100，学习率：5.0e-04，正则化：1.0e-01，验证准确率：53.40%
隐藏单元：100，学习率：1.0e-03，正则化：0.0e+00，验证准确率：52.30%
隐藏单元：100，学习率：1.0e-03，正则化：1.0e-01，验证准确率：51.90%
隐藏单元：200，学习率：5.0e-04，正则化：0.0e+00，验证准确率：52.50%
隐藏单元：200，学习率：5.0e-04，正则化：1.0e-01，验证准确率：52.70%
隐藏单元：200，学习率：1.0e-03，正则化：0.0e+00，验证准确率：53.40%
隐藏单元：200，学习率：1.0e-03，正则化：1.0e-01，验证准确率：53.30%

最佳验证准确率：53.40%
```

Test your model!

Run your best model on the validation and test sets. You should achieve above 48% accuracy on the validation set and the test set.

In [17]:
```
y_val_pred = np.argmax(best_model.loss(data['X_val']), axis=1)
print('Validation set accuracy: ', (y_val_pred == data['y_val']).mean())
```

```
Validation set accuracy:  0.534
```

In [18]:
```
y_test_pred = np.argmax(best_model.loss(data['X_test']), axis=1)
print('Test set accuracy: ', (y_test_pred == data['y_test']).mean())
```

```
Test set accuracy:  0.519
```

In [19]:
```
# Save best model
best_model.save("best_two_layer_net.npy")
```

```
best_two_layer_net.npy saved.
```

Inline Question 2:

Now that you have trained a Neural Network classifier, you may find that your testing accuracy is much lower than the training accuracy. In what ways can we decrease this gap? Select all that apply.

1. Train on a larger dataset.

2. Add more hidden units.
3. Increase the regularization strength.
4. None of the above.

$\color{blue}{\textit Your Answer:}$1、3

$\color{blue}{\textit Your Explanation:}$训练准确率明显高于测试准确率，通常说明模型存在过拟合。

1. 增大训练数据集：更多有代表性的训练样本能帮助模型学习普遍规律，减少过拟合。
2. 增加隐藏单元：会提高模型容量，可能加重过拟合，因此不选。
3. 增强正则化：限制模型复杂度，减少对训练数据细节和噪声的拟合，有助于缩小训练与测试准确率的差距。

In [ ]:

Generative AI Use: For the purposes of the assignments, the use of generative AI is subject to the same policies regarding collaboration. Just as with other collaborators, each student must write down the solutions independently of the output of the interaction and the submission should include a note denoting the nature of the collaboration. The use of generative AI tools to substantially complete sections of the assignments is not in line with the spirit of the assignments, and would be a violation of the Honor Code.

This mounts your Google Drive to the Colab VM.

from google.colab import drive drive.mount('/content/drive')

TODO: Enter the foldername in your Drive where you have saved the unzipped

assignment folder, e.g. 'cs231n/assignments/assignment1/'

FOLDERNAME = 'cs231n/assignments/assignment1/' assert FOLDERNAME is not None, "[!] Enter the foldername."

Now that we've mounted your Drive, this ensures that

the Python interpreter of the Colab VM can load

python files from within it.

import sys sys.path.append('/content/drive/My Drive/{}'.format(FOLDERNAME))

This downloads the CIFAR-10 dataset to your Drive

if it doesn't already exist.

%cd /content/drive/My\ Drive/$FOLDERNAME/cs231n/datasets/ !bash get_datasets.sh %cd /content/drive/My\ Drive/$FOLDERNAME

Image features exercise

Complete and hand in this completed worksheet (including its outputs and any supporting code outside of the worksheet) with your assignment submission. For more details see the assignments page on the course website.

We have seen that we can achieve reasonable performance on an image classification task by training a linear classifier on the pixels of the input image. In this exercise we will show that we can improve our classification performance by training linear classifiers not on raw pixels but on features that are computed from the raw pixels.

All of your work for this exercise will be done in this notebook.

In [1]:
```
import random
import numpy as np
from cs231n.data_utils import load_CIFAR10
import matplotlib.pyplot as plt


%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# for auto-reloading extenrnal modules
# see http://stackoverflow.com/questions/1907993/autoreload-of-modules-in-ipytho
%load_ext autoreload
%autoreload 2
```

Load data

Similar to previous exercises, we will load CIFAR-10 data from disk.

In [2]:
```
from cs231n.features import color_histogram_hsv, hog_feature

def get_CIFAR10_data(num_training=49000, num_validation=1000, num_test=1000):
    # Load the raw CIFAR-10 data
    cifar10_dir = 'cs231n/datasets/cifar-10-batches-py'

    # Cleaning up variables to prevent loading data multiple times (which may ca
    try:
       del X_train, y_train
       del X_test, y_test
       print('Clear previously loaded data.')
    except:
```

```
        pass

    X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

    # Subsample the data
    mask = list(range(num_training, num_training + num_validation))
    X_val = X_train[mask]
    y_val = y_train[mask]
    mask = list(range(num_training))
    X_train = X_train[mask]
    y_train = y_train[mask]
    mask = list(range(num_test))
    X_test = X_test[mask]
    y_test = y_test[mask]

    return X_train, y_train, X_val, y_val, X_test, y_test

X_train, y_train, X_val, y_val, X_test, y_test = get_CIFAR10_data()
```

In [3]:
```
%pip install future
```

```
Requirement already satisfied: future in E:\python\Lib\site-packages (1.0.0)
Note: you may need to restart the kernel to use updated packages.
[notice] A new release of pip is available: 26.2.1 -> 26.2.2
[notice] To update, run: E:\python\python.exe -m pip install --upgrade pip
```

Extract Features

For each image we will compute a Histogram of Oriented Gradients (HOG) as well as a color histogram using the hue channel in HSV color space. We form our final feature vector for each image by concatenating the HOG and color histogram feature vectors.

Roughly speaking, HOG should capture the texture of the image while ignoring color information, and the color histogram represents the color of the input image while ignoring texture. As a result, we expect that using both together ought to work better than using either alone. Verifying this assumption would be a good thing to try for your own interest.

The `hog_feature` and `color_histogram_hsv` functions both operate on a single image and return a feature vector for that image. The extract_features function takes a set of images and a list of feature functions and evaluates each feature function on each image, storing the results in a matrix where each column is the concatenation of all feature vectors for a single image.

In [4]:
```
from cs231n.features import *

# num_color_bins = 10 # Number of bins in the color histogram
num_color_bins = 25 # Number of bins in the color histogram
feature_fns = [hog_feature, lambda img: color_histogram_hsv(img, nbin=num_color_
X_train_feats = extract_features(X_train, feature_fns, verbose=True)
X_val_feats = extract_features(X_val, feature_fns)
X_test_feats = extract_features(X_test, feature_fns)
```

```
# Preprocessing: Subtract the mean feature
mean_feat = np.mean(X_train_feats, axis=0, keepdims=True)
X_train_feats -= mean_feat
X_val_feats -= mean_feat
X_test_feats -= mean_feat

# Preprocessing: Divide by standard deviation. This ensures that each feature
# has roughly the same scale.
std_feat = np.std(X_train_feats, axis=0, keepdims=True)
X_train_feats /= std_feat
X_val_feats /= std_feat
X_test_feats /= std_feat

# Preprocessing: Add a bias dimension
X_train_feats = np.hstack([X_train_feats, np.ones((X_train_feats.shape[0], 1))])
X_val_feats = np.hstack([X_val_feats, np.ones((X_val_feats.shape[0], 1))])
X_test_feats = np.hstack([X_test_feats, np.ones((X_test_feats.shape[0], 1))])
```

```
Done extracting features for 1000 / 49000 images
Done extracting features for 2000 / 49000 images
Done extracting features for 3000 / 49000 images
Done extracting features for 4000 / 49000 images
Done extracting features for 5000 / 49000 images
Done extracting features for 6000 / 49000 images
Done extracting features for 7000 / 49000 images
Done extracting features for 8000 / 49000 images
Done extracting features for 9000 / 49000 images
Done extracting features for 10000 / 49000 images
Done extracting features for 11000 / 49000 images
Done extracting features for 12000 / 49000 images
Done extracting features for 13000 / 49000 images
Done extracting features for 14000 / 49000 images
Done extracting features for 15000 / 49000 images
Done extracting features for 16000 / 49000 images
Done extracting features for 17000 / 49000 images
Done extracting features for 18000 / 49000 images
Done extracting features for 19000 / 49000 images
Done extracting features for 20000 / 49000 images
Done extracting features for 21000 / 49000 images
Done extracting features for 22000 / 49000 images
Done extracting features for 23000 / 49000 images
Done extracting features for 24000 / 49000 images
Done extracting features for 25000 / 49000 images
Done extracting features for 26000 / 49000 images
Done extracting features for 27000 / 49000 images
Done extracting features for 28000 / 49000 images
Done extracting features for 29000 / 49000 images
Done extracting features for 30000 / 49000 images
Done extracting features for 31000 / 49000 images
Done extracting features for 32000 / 49000 images
Done extracting features for 33000 / 49000 images
Done extracting features for 34000 / 49000 images
Done extracting features for 35000 / 49000 images
Done extracting features for 36000 / 49000 images
Done extracting features for 37000 / 49000 images
Done extracting features for 38000 / 49000 images
Done extracting features for 39000 / 49000 images
Done extracting features for 40000 / 49000 images
Done extracting features for 41000 / 49000 images
Done extracting features for 42000 / 49000 images
Done extracting features for 43000 / 49000 images
Done extracting features for 44000 / 49000 images
Done extracting features for 45000 / 49000 images
Done extracting features for 46000 / 49000 images
Done extracting features for 47000 / 49000 images
Done extracting features for 48000 / 49000 images
Done extracting features for 49000 / 49000 images
```

Train Softmax classifier on features

Using the Softmax code developed earlier in the assignment, train Softmax classifiers on top of the features extracted above; this should achieve better results than training them directly on top of raw pixels.

In [5]:
```
import numpy as np
from cs231n.classifiers.linear_classifier import Softmax
```

```python
# 调整学习率和正则化强度的范围
learning_rates = [1e-3, 2e-2, 3e-2]
regularization_strengths = [1e-4, 5e-4, 1e-3]

results = {}
best_val = -1
best_softmax = None

for lr in learning_rates:
    for reg in regularization_strengths:

        # Initialize Softmax classifier
        softmax = Softmax()

        # Train on extracted features
        softmax.train(
            X_train_feats,
            y_train,
            learning_rate=lr,
            reg=reg,
            num_iters=2500,
            verbose=False
        )

        # Training accuracy
        y_train_pred = softmax.predict(X_train_feats)
        train_accuracy = np.mean(y_train_pred == y_train)

        # Validation accuracy
        y_val_pred = softmax.predict(X_val_feats)
        val_accuracy = np.mean(y_val_pred == y_val)

        # Save results
        results[(lr, reg)] = (train_accuracy, val_accuracy)

        # Update best classifier
        if val_accuracy > best_val:
            best_val = val_accuracy
            best_softmax = softmax

# Print out results.
for lr, reg in sorted(results):
    train_accuracy, val_accuracy = results[(lr, reg)]
    print('lr %e reg %e train accuracy: %f val accuracy: %f' % (
        lr, reg, train_accuracy, val_accuracy))

print('best validation accuracy achieved: %f' % best_val)
```

```
lr 1.000000e-03 reg 1.000000e-04 train accuracy: 0.465592 val accuracy: 0.473000
lr 1.000000e-03 reg 5.000000e-04 train accuracy: 0.465796 val accuracy: 0.477000
lr 1.000000e-03 reg 1.000000e-03 train accuracy: 0.465184 val accuracy: 0.472000
lr 2.000000e-02 reg 1.000000e-04 train accuracy: 0.525755 val accuracy: 0.521000
lr 2.000000e-02 reg 5.000000e-04 train accuracy: 0.524000 val accuracy: 0.522000
lr 2.000000e-02 reg 1.000000e-03 train accuracy: 0.525653 val accuracy: 0.528000
lr 3.000000e-02 reg 1.000000e-04 train accuracy: 0.526265 val accuracy: 0.526000
lr 3.000000e-02 reg 5.000000e-04 train accuracy: 0.524694 val accuracy: 0.520000
lr 3.000000e-02 reg 1.000000e-03 train accuracy: 0.526633 val accuracy: 0.530000
best validation accuracy achieved: 0.530000
```

In [6]:
```
# Evaluate your trained Softmax on the test set: you should be able to get at le
y_test_pred = best_softmax.predict(X_test_feats)
test_accuracy = np.mean(y_test == y_test_pred)
print(test_accuracy)
```

```
0.516
```

In [7]:
```
# Save best softmax model
best_softmax.save("best_softmax_features.npy")
```

```
best_softmax_features.npy saved.
```

In [8]:
```
examples_per_class = 8

classes = ['plane', 'car', 'bird', 'cat', 'deer',
           'dog', 'frog', 'horse', 'ship', 'truck']

plt.figure(figsize=(15, 12))

for cls, cls_name in enumerate(classes):

    idxs = np.where((y_test != cls) & (y_test_pred == cls))[0]

    # 检查当前类别是否有错误分类的图片
    if len(idxs) == 0:
        print(f"{cls_name}: 没有找到错误分类的图片")
        continue

    # 如果不足8张，就选择实际存在的数量
    num_samples = min(len(idxs), examples_per_class)

    idxs = np.random.choice(idxs, num_samples, replace=False)

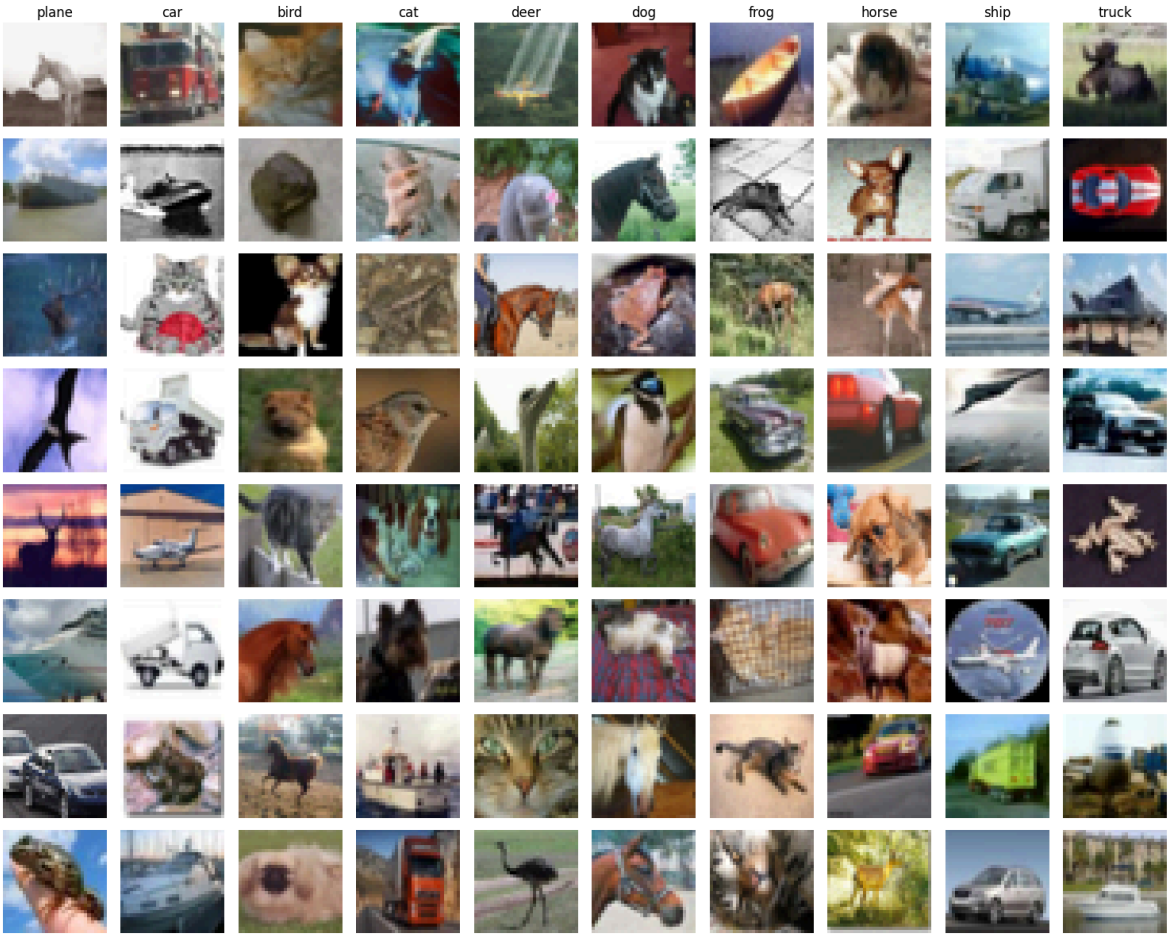
    for i, idx in enumerate(idxs):

        plt.subplot(examples_per_class, len(classes),
                    i * len(classes) + cls + 1)

        plt.imshow(X_test[idx].astype('uint8'))
        plt.axis('off')

        if i == 0:
            plt.title(cls_name)

plt.tight_layout()
plt.show()
```



Inline question 1:

Describe the misclassification results that you see. Do they make sense?

$\color{blue}{\textit Your Answer:}$ 这些错误分类结果总体上是合理的。 模型容易混淆具有相似视觉特征的类别，例如猫和狗、汽车和卡车。 有些图片由于分辨率较低、背景复杂，或者颜色和形状相似，因此很难正确分类。 由于 CIFAR-10 数据集中的图片尺寸只有 32×32 像素，模型有时难以区分不同的物体。 因此，这些错误分类是可以理解的。

Neural Network on image features

Earlier in this assigment we saw that training a two-layer neural network on raw pixels achieved better classification performance than linear classifiers on raw pixels. In this notebook we have seen that linear classifiers on image features outperform linear classifiers on raw pixels.

For completeness, we should also try training a neural network on image features. This approach should outperform all previous approaches: you should easily be able to achieve over 55% classification accuracy on the test set; our best model achieves about 60% classification accuracy.

In [9]:
```
# Preprocessing: Remove the bias dimension
# Make sure to run this cell only ONCE
print(X_train_feats.shape)
X_train_feats = X_train_feats[:, :-1]
```

```
X_val_feats = X_val_feats[:, :-1]
X_test_feats = X_test_feats[:, :-1]

print(X_train_feats.shape)
```

```
(49000, 170)
(49000, 169)
```

In [10]:

```
from cs231n.classifiers.fc_net import TwoLayerNet
from cs231n.solver import Solver

input_dim = X_train_feats.shape[1]
hidden_dim = 500
num_classes = 10

data = {
    'X_train': X_train_feats,
    'y_train': y_train,
    'X_val': X_val_feats,
    'y_val': y_val,
    'X_test': X_test_feats,
    'y_test': y_test,
}

net = TwoLayerNet(input_dim, hidden_dim, num_classes)
best_net = None

################################################################################
# TODO: Train a two-layer neural network on image features. You may want to    #
# cross-validate various parameters as in previous sections. Store your best   #
# model in the best_net variable.                                              #
################################################################################
best_val_acc = -1

# 1. 定义超参数搜索空间
# 学习率和正则化强度通常采用对数尺度进行搜索
learning_rates = [5e-4, 1e-3, 2e-3, 3e-3]
regularization_strengths = [1e-4, 1e-3, 1e-2, 1e-1]

# 2. 循环遍历所有超参数组合
for lr in learning_rates:
    for reg in regularization_strengths:

        # 实例化一个新的两层神经网络模型
        model = TwoLayerNet(input_dim, hidden_dim, num_classes,
                            weight_scale=1e-3, reg=reg)

        # 实例化 Solver
        # 推荐使用 adam 优化器，因为它在特征上收敛得更好、更快
        solver = Solver(model, data,
                        update_rule='adam',
                        optim_config={
                          'learning_rate': lr,
                        },
                        lr_decay=0.95,      # 学习率衰减
                        num_epochs=15,      # 训练轮数
                        batch_size=200,
                        print_every=100,
                        verbose=False)      # 设为False防止输出刷屏
```

```
        # 开始训练模型
        solver.train()

        # 获取模型在验证集上的最高准确率
        val_acc = solver.best_val_acc

        # 打印当前参数的表现
        print(f"lr: {lr:.1e}, reg: {reg:.1e}, val accuracy: {val_acc:.4f}")

        # 3. 如果当前模型表现更好，则更新 best_net
        if val_acc > best_val_acc:
            best_val_acc = val_acc
            # 因为 Solver 训练结束后会自动把模型参数恢复为验证集最好的那一轮
            # 所以直接将 model 赋值给 best_net 即可
            best_net = model

print(f'\nBest validation accuracy achieved: {best_val_acc:.4f}')
```

```
lr: 5.0e-04, reg: 1.0e-04, val accuracy: 0.6180
lr: 5.0e-04, reg: 1.0e-03, val accuracy: 0.6140
lr: 5.0e-04, reg: 1.0e-02, val accuracy: 0.5680
lr: 5.0e-04, reg: 1.0e-01, val accuracy: 0.4530
lr: 1.0e-03, reg: 1.0e-04, val accuracy: 0.6030
lr: 1.0e-03, reg: 1.0e-03, val accuracy: 0.6080
lr: 1.0e-03, reg: 1.0e-02, val accuracy: 0.5780
lr: 1.0e-03, reg: 1.0e-01, val accuracy: 0.4480
lr: 2.0e-03, reg: 1.0e-04, val accuracy: 0.6020
lr: 2.0e-03, reg: 1.0e-03, val accuracy: 0.6120
lr: 2.0e-03, reg: 1.0e-02, val accuracy: 0.5730
lr: 2.0e-03, reg: 1.0e-01, val accuracy: 0.4450
lr: 3.0e-03, reg: 1.0e-04, val accuracy: 0.5900
lr: 3.0e-03, reg: 1.0e-03, val accuracy: 0.6120
lr: 3.0e-03, reg: 1.0e-02, val accuracy: 0.5630
lr: 3.0e-03, reg: 1.0e-01, val accuracy: 0.4510

Best validation accuracy achieved: 0.6180
```

In [11]:
```
# Run your best neural net classifier on the test set. You should be able
# to get more than 58% accuracy. It is also possible to get >60% accuracy
# with careful tuning.

y_test_pred = np.argmax(best_net.loss(data['X_test']), axis=1)
test_acc = (y_test_pred == data['y_test']).mean()
print(test_acc)
```

```
0.585
```

In [12]:
```
# Save best model
best_net.save("best_two_layer_net_features.npy")
```

```
best_two_layer_net_features.npy saved.
```

In [ ]:

Generative AI Use: For the purposes of the assignments, the use of generative AI is subject to the same policies regarding collaboration. Just as with other collaborators, each student must write down the solutions independently of the output of the interaction and the submission should include a note denoting the nature of the collaboration. The use of generative AI tools to substantially complete sections of the assignments is not in line with the spirit of the assignments, and would be a violation of the Honor Code.

In [1]:
```
# This mounts your Google Drive to the Colab VM.
from google.colab import drive
drive.mount('/content/drive')

# TODO: Enter the foldername in your Drive where you have saved the unzipped
# assignment folder, e.g. 'cs231n/assignments/assignment1/'
FOLDERNAME = 'cs231n/assignments/assignment1/'
assert FOLDERNAME is not None, "[!] Enter the foldername."

# Now that we've mounted your Drive, this ensures that
# the Python interpreter of the Colab VM can load
# python files from within it.
import sys
sys.path.append('/content/drive/My Drive/{}'.format(FOLDERNAME))

# This downloads the CIFAR-10 dataset to your Drive
# if it doesn't already exist.
%cd /content/drive/My\ Drive/$FOLDERNAME/cs231n/datasets/
# %cd /content/drive/My\ Drive/$FOLDERNAME
!bash get_datasets.sh
%cd /content/drive/My\ Drive/$FOLDERNAME
```

```
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
Cell In[1], line 2
      1 # This mounts your Google Drive to the Colab VM.
----> 2 from google.colab import drive
      3 drive.mount('/content/drive')
      4
      5 # TODO: Enter the foldername in your Drive where you have saved the unzip
ped

ModuleNotFoundError: No module named 'google'
```

Multi-Layer Fully Connected Network

In this exercise, you will implement a fully connected network with an arbitrary number of hidden layers.

In [ ]:
```
# from google.colab import drive
# drive.mount('/content/drive')
```

Read through the `FullyConnectedNet` class in the file `cs231n/classifiers/fc_net.py`.

Implement the network initialization, forward pass, and backward pass. Throughout this assignment, you will be implementing layers in `cs231n/layers.py`. You can re-use your implementations for `affine_forward`, `affine_backward`, `relu_forward`, `relu_backward`, and `softmax_loss` from before. For right now, don't worry about implementing dropout or batch/layer normalization yet, as you will add those features later.

In [1]:
```
# Setup cell.
import time
import numpy as np
import matplotlib.pyplot as plt
from cs231n.classifiers.fc_net import *
from cs231n.data_utils import get_CIFAR10_data
from cs231n.gradient_check import eval_numerical_gradient, eval_numerical_gradie
from cs231n.solver import Solver

%matplotlib inline
plt.rcParams["figure.figsize"] = (10.0, 8.0)  # Set default size of plots.
plt.rcParams["image.interpolation"] = "nearest"
plt.rcParams["image.cmap"] = "gray"

%load_ext autoreload
%autoreload 2

def rel_error(x, y):
    """Returns relative error."""
    return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y))))
```

In [2]:
```
# Load the (preprocessed) CIFAR-10 data.
data = get_CIFAR10_data()
for k, v in list(data.items()):
    print(f"{k}: {v.shape}")
```

```
X_train: (49000, 3, 32, 32)
y_train: (49000,)
X_val: (1000, 3, 32, 32)
y_val: (1000,)
X_test: (1000, 3, 32, 32)
y_test: (1000,)
```

Initial Loss and Gradient Check

As a sanity check, run the following to check the initial loss and to gradient check the network both with and without regularization. This is a good way to see if the initial losses seem reasonable.

For gradient checking, you should expect to see errors around 1e-7 or less.

In [3]:
```
np.random.seed(231)
N, D, H1, H2, C = 2, 15, 20, 30, 10
X = np.random.randn(N, D)
y = np.random.randint(C, size=(N,))

for reg in [0, 3.14]:
    print("Running check with reg = ", reg)
```

```
    model = FullyConnectedNet(
        [H1, H2],
        input_dim=D,
        num_classes=C,
        reg=reg,
        weight_scale=5e-2,
        dtype=np.float64
    )

    loss, grads = model.loss(X, y)
    print("Initial loss: ", loss)

    # Most of the errors should be on the order of e-7 or smaller.
    # NOTE: It is fine however to see an error for W2 on the order of e-5
    # for the check when reg = 0.0
    for name in sorted(grads):
        f = lambda _: model.loss(X, y)[0]
        grad_num = eval_numerical_gradient(f, model.params[name], verbose=False,
        print(f"{name} relative error: {rel_error(grad_num, grads[name])}")
```

```
Running check with reg =  0
Initial loss:  0.0
Running check with reg =  3.14
Initial loss:  0.0
```

As another sanity check, make sure your network can overfit on a small dataset of 50 images. First, we will try a three-layer network with 100 units in each hidden layer. In the following cell, tweak the **learning rate** and **weight initialization scale** to overfit and achieve 100% training accuracy within 20 epochs.

In [5]:
```
# TODO: Use a three-layer Net to overfit 50 training examples by
# tweaking just the learning rate and initialization scale.

num_train = 50
small_data = {
  "X_train": data["X_train"][:num_train],
  "y_train": data["y_train"][:num_train],
  "X_val": data["X_val"],
  "y_val": data["y_val"],
}

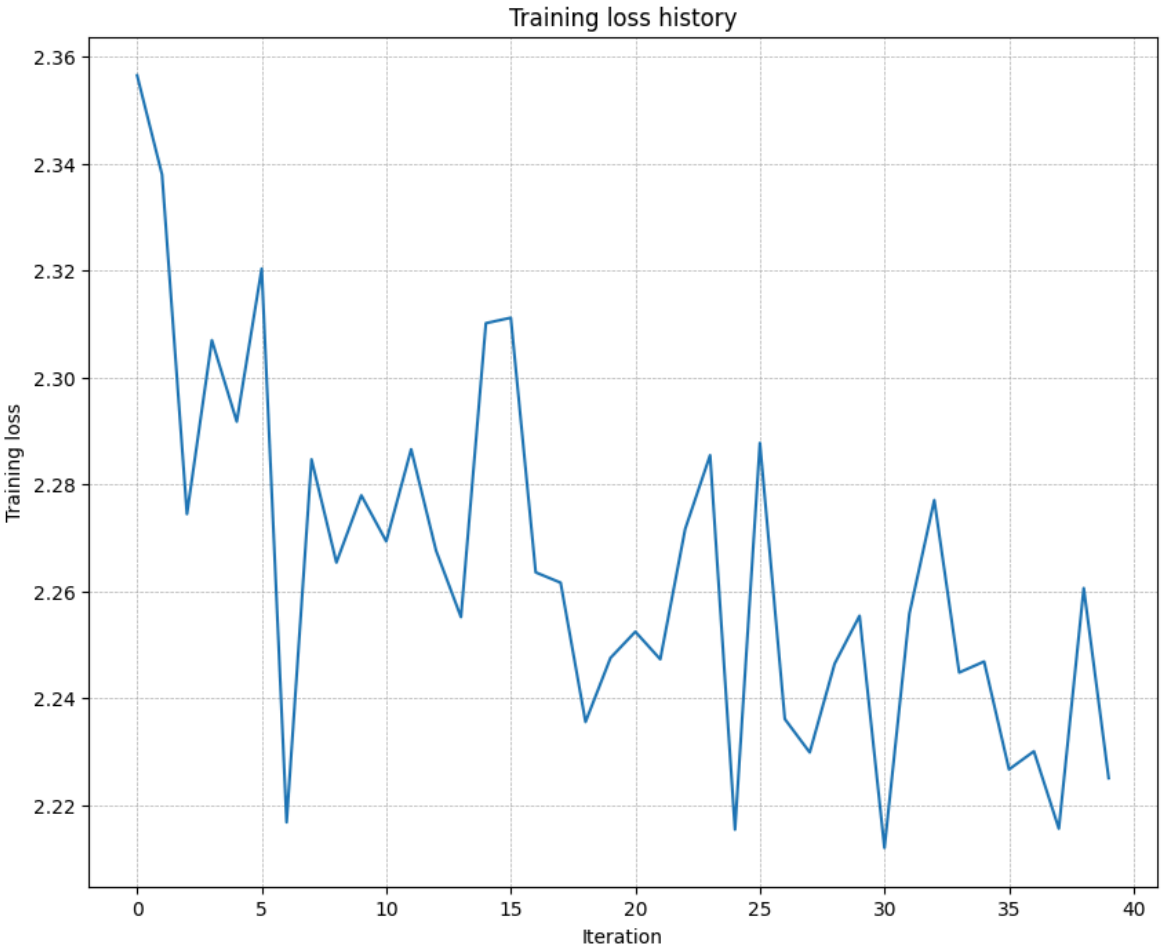
weight_scale = 1e-2   # Experiment with this!
learning_rate = 1e-4  # Experiment with this!


model = FullyConnectedNet(
    [100, 100],
    weight_scale=weight_scale,
    dtype=np.float64
)
solver = Solver(
    model,
    small_data,
    print_every=10,
    num_epochs=20,
    batch_size=25,
    update_rule="sgd",
    optim_config={"learning_rate": learning_rate},
)
```

```
solver.train()

plt.plot(solver.loss_history)
plt.title("Training loss history")
plt.xlabel("Iteration")
plt.ylabel("Training loss")
plt.grid(linestyle='--', linewidth=0.5)
plt.show()
```

```
(Iteration 1 / 40) loss: 2.356451
(Epoch 0 / 20) train acc: 0.120000; val_acc: 0.110000
(Epoch 1 / 20) train acc: 0.120000; val_acc: 0.110000
(Epoch 2 / 20) train acc: 0.120000; val_acc: 0.111000
(Epoch 3 / 20) train acc: 0.120000; val_acc: 0.113000
(Epoch 4 / 20) train acc: 0.120000; val_acc: 0.115000
(Epoch 5 / 20) train acc: 0.120000; val_acc: 0.114000
(Iteration 11 / 40) loss: 2.269347
(Epoch 6 / 20) train acc: 0.120000; val_acc: 0.113000
(Epoch 7 / 20) train acc: 0.120000; val_acc: 0.115000
(Epoch 8 / 20) train acc: 0.120000; val_acc: 0.116000
(Epoch 9 / 20) train acc: 0.120000; val_acc: 0.116000
(Epoch 10 / 20) train acc: 0.120000; val_acc: 0.113000
(Iteration 21 / 40) loss: 2.252458
(Epoch 11 / 20) train acc: 0.120000; val_acc: 0.116000
(Epoch 12 / 20) train acc: 0.120000; val_acc: 0.115000
(Epoch 13 / 20) train acc: 0.120000; val_acc: 0.117000
(Epoch 14 / 20) train acc: 0.120000; val_acc: 0.116000
(Epoch 15 / 20) train acc: 0.120000; val_acc: 0.118000
(Iteration 31 / 40) loss: 2.212044
(Epoch 16 / 20) train acc: 0.140000; val_acc: 0.119000
(Epoch 17 / 20) train acc: 0.160000; val_acc: 0.120000
(Epoch 18 / 20) train acc: 0.160000; val_acc: 0.124000
(Epoch 19 / 20) train acc: 0.160000; val_acc: 0.124000
(Epoch 20 / 20) train acc: 0.160000; val_acc: 0.127000
```



Now, try to use a five-layer network with 100 units on each layer to overfit on 50 training examples. Again, you will have to adjust the learning rate and weight initialization scale, but you should be able to achieve 100% training accuracy within 20 epochs.

In [6]:
```
# TODO: Use a five-layer Net to overfit 50 training examples by
# tweaking just the learning rate and initialization scale.

num_train = 50
small_data = {
  'X_train': data['X_train'][:num_train],
  'y_train': data['y_train'][:num_train],
  'X_val': data['X_val'],
  'y_val': data['y_val'],
}

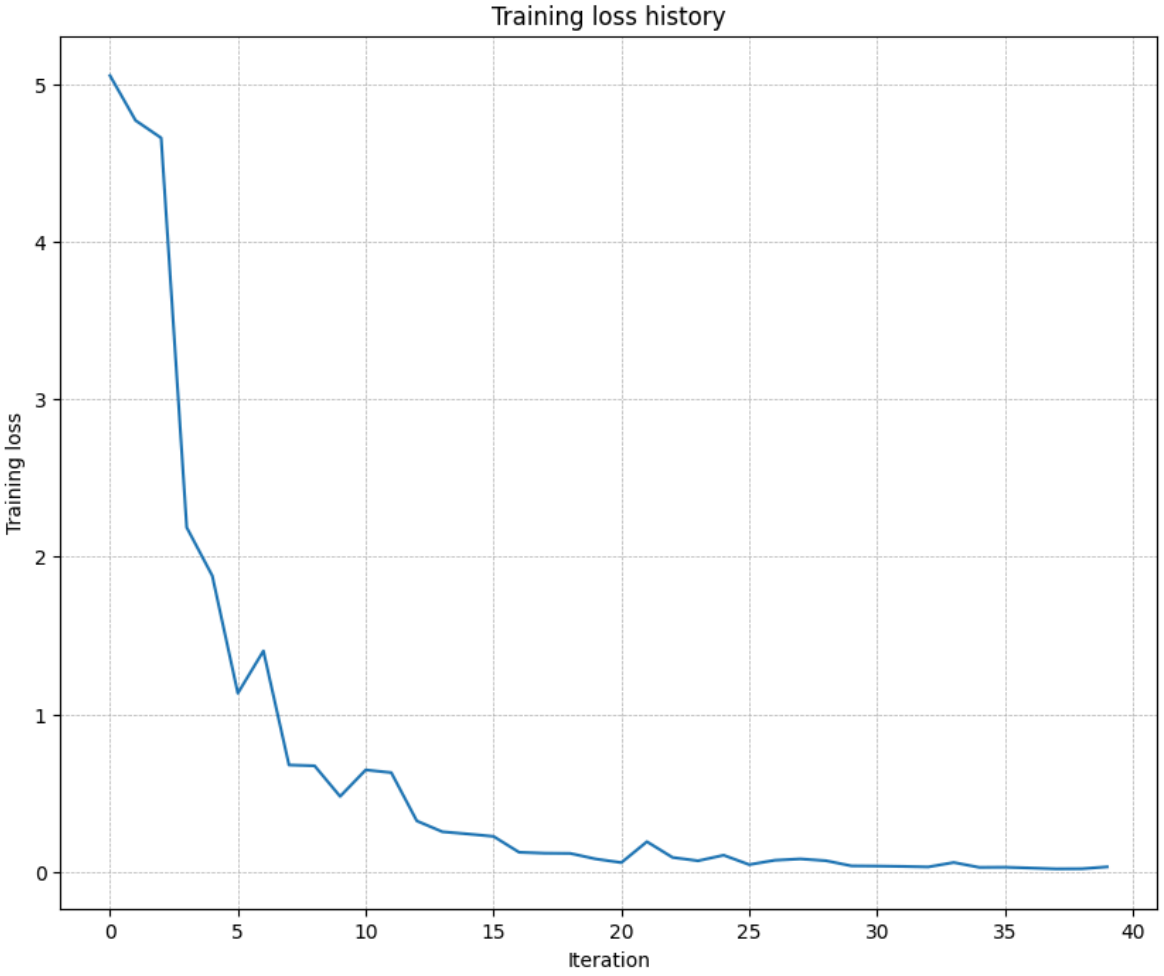
np.random.seed(231)
learning_rate = 1e-2
weight_scale = 5e-2


model = FullyConnectedNet(
    [100, 100, 100, 100],
    weight_scale=weight_scale,
    dtype=np.float64
)
solver = Solver(
    model,
    small_data,
```

```
    print_every=10,
    num_epochs=20,
    batch_size=25,
    update_rule='sgd',
    optim_config={'learning_rate': learning_rate},
)
solver.train()

plt.plot(solver.loss_history)
plt.title('Training loss history')
plt.xlabel('Iteration')
plt.ylabel('Training loss')
plt.grid(linestyle='--', linewidth=0.5)
plt.show()
```

```
(Iteration 1 / 40) loss: 5.054391
(Epoch 0 / 20) train acc: 0.120000; val_acc: 0.079000
(Epoch 1 / 20) train acc: 0.380000; val_acc: 0.152000
(Epoch 2 / 20) train acc: 0.360000; val_acc: 0.115000
(Epoch 3 / 20) train acc: 0.820000; val_acc: 0.139000
(Epoch 4 / 20) train acc: 0.900000; val_acc: 0.136000
(Epoch 5 / 20) train acc: 0.900000; val_acc: 0.124000
(Iteration 11 / 40) loss: 0.646205
(Epoch 6 / 20) train acc: 0.980000; val_acc: 0.133000
(Epoch 7 / 20) train acc: 1.000000; val_acc: 0.128000
(Epoch 8 / 20) train acc: 1.000000; val_acc: 0.138000
(Epoch 9 / 20) train acc: 1.000000; val_acc: 0.124000
(Epoch 10 / 20) train acc: 1.000000; val_acc: 0.127000
(Iteration 21 / 40) loss: 0.057810
(Epoch 11 / 20) train acc: 1.000000; val_acc: 0.125000
(Epoch 12 / 20) train acc: 1.000000; val_acc: 0.132000
(Epoch 13 / 20) train acc: 1.000000; val_acc: 0.132000
(Epoch 14 / 20) train acc: 1.000000; val_acc: 0.133000
(Epoch 15 / 20) train acc: 1.000000; val_acc: 0.131000
(Iteration 31 / 40) loss: 0.035820
(Epoch 16 / 20) train acc: 1.000000; val_acc: 0.129000
(Epoch 17 / 20) train acc: 1.000000; val_acc: 0.132000
(Epoch 18 / 20) train acc: 1.000000; val_acc: 0.128000
(Epoch 19 / 20) train acc: 1.000000; val_acc: 0.126000
(Epoch 20 / 20) train acc: 1.000000; val_acc: 0.130000
```



Inline Question 1:

Did you notice anything about the comparative difficulty of training the three-layer network vs. training the five-layer network? In particular, based on your experience, which network seemed more sensitive to the initialization scale? Why do you think that is the case?

Answer:

五层网络比三层网络更难训练，对权重初始化尺度更敏感。因为网络更深，前向传播和反向传播都要经过更多次矩阵乘法。初始化权重过小时，信号和梯度容易逐层衰减，导致学习缓慢；初始化权重过大时，又可能导致梯度过大和训练不稳定。因此，五层网络需要更仔细地调整初始化尺度和学习率。

Update rules

So far we have used vanilla stochastic gradient descent (SGD) as our update rule. More sophisticated update rules can make it easier to train deep networks. We will implement a few of the most commonly used update rules and compare them to vanilla SGD.

SGD+Momentum

Stochastic gradient descent with momentum is a widely used update rule that tends to make deep networks converge faster than vanilla stochastic gradient descent. See the Momentum Update section at http://cs231n.github.io/neural-networks-3/#sgd for more information.

Open the file `cs231n/optim.py` and read the documentation at the top of the file to make sure you understand the API. Implement the SGD+momentum update rule in the function `sgd_momentum` and run the following to check your implementation. You should see errors less than e-8.

In [7]:
```
from cs231n.optim import sgd_momentum

N, D = 4, 5
w = np.linspace(-0.4, 0.6, num=N*D).reshape(N, D)
dw = np.linspace(-0.6, 0.4, num=N*D).reshape(N, D)
v = np.linspace(0.6, 0.9, num=N*D).reshape(N, D)

config = {"learning_rate": 1e-3, "velocity": v}
next_w, _ = sgd_momentum(w, dw, config=config)

expected_next_w = np.asarray([
  [ 0.1406,      0.20738947,  0.27417895,  0.34096842,  0.40775789],
  [ 0.47454737,  0.54133684,  0.60812632,  0.67491579,  0.74170526],
  [ 0.80849474,  0.87528421,  0.94207368,  1.00886316,  1.07565263],
  [ 1.14244211,  1.20923158,  1.27602105,  1.34281053,  1.4096    ]])
expected_velocity = np.asarray([
  [ 0.5406,      0.55475789,  0.56891579, 0.58307368,  0.59723158],
  [ 0.61138947,  0.62554737,  0.63970526,  0.65386316,  0.66802105],
  [ 0.68217895,  0.69633684,  0.71049474,  0.72465263,  0.73881053],
  [ 0.75296842,  0.76712632,  0.78128421,  0.79544211,  0.8096    ]])

# Should see relative errors around e-8 or less
print("next_w error: ", rel_error(next_w, expected_next_w))
print("velocity error: ", rel_error(expected_velocity, config["velocity"]))
```

```
next_w error:  8.882347033505819e-09
velocity error:  4.269287743278663e-09
```

Once you have done so, run the following to train a six-layer network with both SGD and SGD+momentum. You should see the SGD+momentum update rule converge faster.

In [8]:
```
num_train = 4000
small_data = {
  'X_train': data['X_train'][:num_train],
  'y_train': data['y_train'][:num_train],
  'X_val': data['X_val'],
  'y_val': data['y_val'],
}

solvers = {}

for update_rule in ['sgd', 'sgd_momentum']:
```

```
    print('Running with ', update_rule)
    model = FullyConnectedNet(
        [100, 100, 100, 100, 100],
        weight_scale=5e-2
    )

    solver = Solver(
        model,
        small_data,
        num_epochs=5,
        batch_size=100,
        update_rule=update_rule,
        optim_config={'learning_rate': 5e-3},
        verbose=True,
    )
    solvers[update_rule] = solver
    solver.train()

fig, axes = plt.subplots(3, 1, figsize=(15, 15))

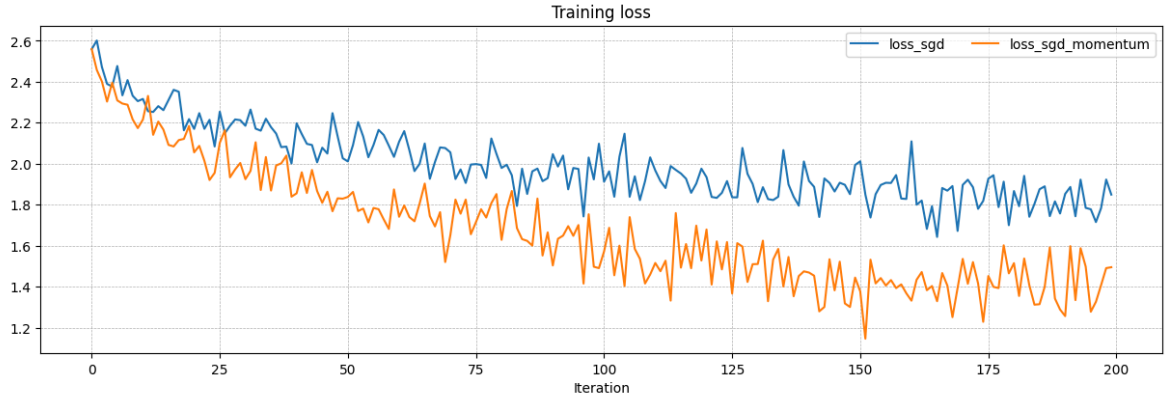
axes[0].set_title('Training loss')
axes[0].set_xlabel('Iteration')
axes[1].set_title('Training accuracy')
axes[1].set_xlabel('Epoch')
axes[2].set_title('Validation accuracy')
axes[2].set_xlabel('Epoch')

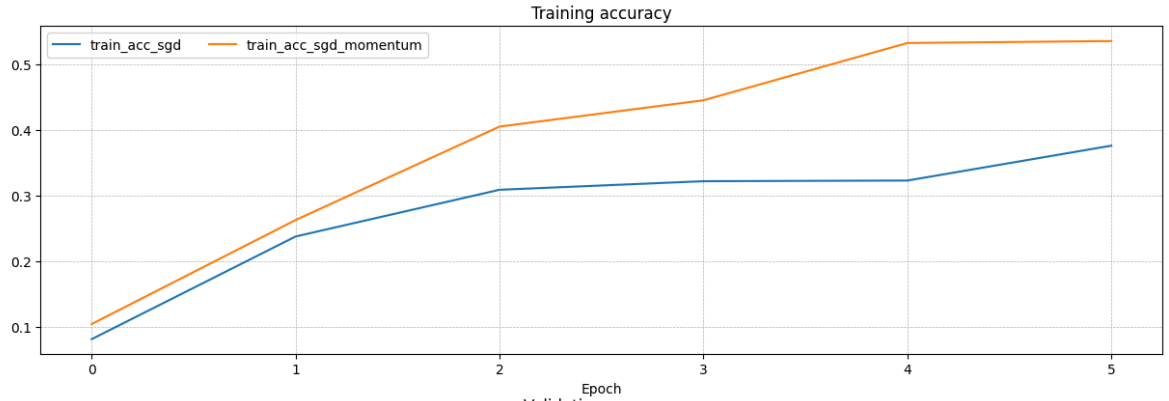
for update_rule, solver in solvers.items():
    axes[0].plot(solver.loss_history, label=f"loss_{update_rule}")
    axes[1].plot(solver.train_acc_history, label=f"train_acc_{update_rule}")
    axes[2].plot(solver.val_acc_history, label=f"val_acc_{update_rule}")

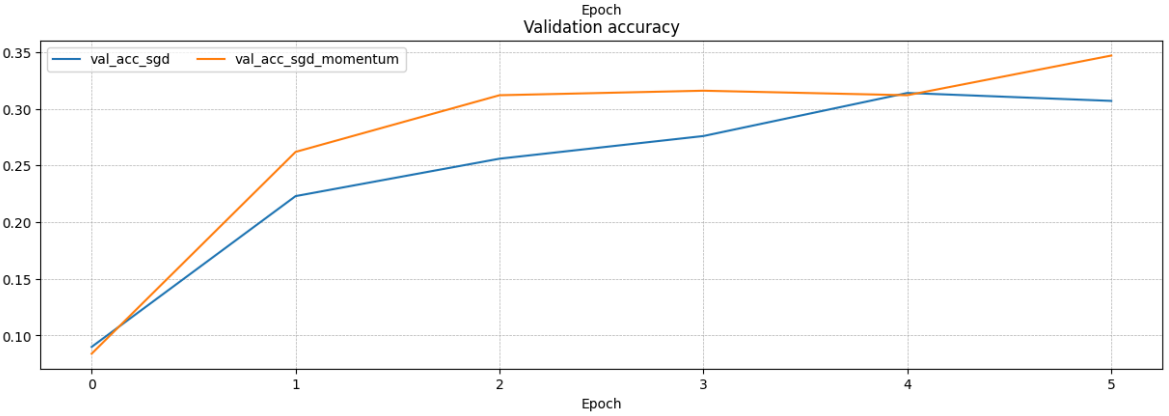
for ax in axes:
    ax.legend(loc="best", ncol=4)
    ax.grid(linestyle='--', linewidth=0.5)

plt.show()
```

```
Running with  sgd
(Iteration 1 / 200) loss: 2.559169
(Epoch 0 / 5) train acc: 0.082000; val_acc: 0.090000
(Iteration 11 / 200) loss: 2.316312
(Iteration 21 / 200) loss: 2.170233
(Iteration 31 / 200) loss: 2.185428
(Epoch 1 / 5) train acc: 0.238000; val_acc: 0.223000
(Iteration 41 / 200) loss: 2.197252
(Iteration 51 / 200) loss: 2.011295
(Iteration 61 / 200) loss: 2.107506
(Iteration 71 / 200) loss: 2.055619
(Epoch 2 / 5) train acc: 0.309000; val_acc: 0.256000
(Iteration 81 / 200) loss: 1.979278
(Iteration 91 / 200) loss: 2.046660
(Iteration 101 / 200) loss: 1.912348
(Iteration 111 / 200) loss: 1.966966
(Epoch 3 / 5) train acc: 0.322000; val_acc: 0.276000
(Iteration 121 / 200) loss: 1.934577
(Iteration 131 / 200) loss: 1.812541
(Iteration 141 / 200) loss: 1.915866
(Iteration 151 / 200) loss: 2.012019
(Epoch 4 / 5) train acc: 0.323000; val_acc: 0.314000
(Iteration 161 / 200) loss: 2.109186
(Iteration 171 / 200) loss: 1.896687
(Iteration 181 / 200) loss: 1.866722
(Iteration 191 / 200) loss: 1.852930
(Epoch 5 / 5) train acc: 0.376000; val_acc: 0.307000
Running with  sgd_momentum
(Iteration 1 / 200) loss: 2.559628
(Epoch 0 / 5) train acc: 0.105000; val_acc: 0.084000
(Iteration 11 / 200) loss: 2.215810
(Iteration 21 / 200) loss: 2.055531
(Iteration 31 / 200) loss: 1.924695
(Epoch 1 / 5) train acc: 0.263000; val_acc: 0.262000
(Iteration 41 / 200) loss: 1.855510
(Iteration 51 / 200) loss: 1.837978
(Iteration 61 / 200) loss: 1.741062
(Iteration 71 / 200) loss: 1.651159
(Epoch 2 / 5) train acc: 0.405000; val_acc: 0.312000
(Iteration 81 / 200) loss: 1.628412
(Iteration 91 / 200) loss: 1.503412
(Iteration 101 / 200) loss: 1.571853
(Iteration 111 / 200) loss: 1.515459
(Epoch 3 / 5) train acc: 0.445000; val_acc: 0.316000
(Iteration 121 / 200) loss: 1.678903
(Iteration 131 / 200) loss: 1.510431
(Iteration 141 / 200) loss: 1.468768
(Iteration 151 / 200) loss: 1.376493
(Epoch 4 / 5) train acc: 0.532000; val_acc: 0.312000
(Iteration 161 / 200) loss: 1.331441
(Iteration 171 / 200) loss: 1.535535
(Iteration 181 / 200) loss: 1.514579
(Iteration 191 / 200) loss: 1.256233
(Epoch 5 / 5) train acc: 0.535000; val_acc: 0.347000
```







RMSProp and Adam

RMSProp [1] and Adam [2] are update rules that set per-parameter learning rates by using a running average of the second moments of gradients.

In the file `cs231n/optim.py`, implement the RMSProp update rule in the `rmsprop` function and implement the Adam update rule in the `adam` function, and check your implementations using the tests below.

NOTE: Please implement the *complete* Adam update rule (with the bias correction mechanism), not the first simplified version mentioned in the course notes.

[1] Tijmen Tieleman and Geoffrey Hinton. "Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude." COURSERA: Neural Networks for Machine Learning 4 (2012).

[2] Diederik Kingma and Jimmy Ba, "Adam: A Method for Stochastic Optimization", ICLR 2015.

In [10]:
```
# Test RMSProp implementation
from cs231n.optim import rmsprop

N, D = 4, 5
w = np.linspace(-0.4, 0.6, num=N*D).reshape(N, D)
dw = np.linspace(-0.6, 0.4, num=N*D).reshape(N, D)
cache = np.linspace(0.6, 0.9, num=N*D).reshape(N, D)

config = {'learning_rate': 1e-2, 'cache': cache}
next_w, _ = rmsprop(w, dw, config=config)

expected_next_w = np.asarray([
  [-0.39223849, -0.34037513, -0.28849239, -0.23659121, -0.18467247],
  [-0.132737,   -0.08078555, -0.02881884,  0.02316247,  0.07515774],
  [ 0.12716641,  0.17918792,  0.23122175,  0.28326742,  0.33532447],
  [ 0.38739248,  0.43947102,  0.49155973,  0.54365823,  0.59576619]])
expected_cache = np.asarray([
  [ 0.5976,      0.6126277,   0.6277108,   0.64284931,  0.65804321],
  [ 0.67329252,  0.68859723,  0.70395734,  0.71937285,  0.73484377],
  [ 0.75037008,  0.7659518,   0.78158892,  0.79728144,  0.81302936],
  [ 0.82883269,  0.84469141,  0.86060554,  0.87657507,  0.8926    ]])

# You should see relative errors around e-7 or less
print('next_w error: ', rel_error(expected_next_w, next_w))
print('cache error: ', rel_error(expected_cache, config['cache']))
```

```
next_w error:  9.524687511038133e-08
cache error:  2.6477955807156126e-09
```

In [11]:
```
# Test Adam implementation
from cs231n.optim import adam

N, D = 4, 5
w = np.linspace(-0.4, 0.6, num=N*D).reshape(N, D)
dw = np.linspace(-0.6, 0.4, num=N*D).reshape(N, D)
m = np.linspace(0.6, 0.9, num=N*D).reshape(N, D)
v = np.linspace(0.7, 0.5, num=N*D).reshape(N, D)

config = {'learning_rate': 1e-2, 'm': m, 'v': v, 't': 5}
next_w, _ = adam(w, dw, config=config)

expected_next_w = np.asarray([
  [-0.40094747, -0.34836187, -0.29577703, -0.24319299, -0.19060977],
  [-0.1380274,  -0.08544591, -0.03286534,  0.01971428,  0.0722929],
  [ 0.1248705,   0.17744702,  0.23002243,  0.28259667,  0.33516969],
  [ 0.38774145,  0.44031188,  0.49288093,  0.54544852,  0.59801459]])
expected_v = np.asarray([
  [ 0.69966,     0.68908382,  0.67851319,  0.66794809,  0.65738853,],
  [ 0.64683452,  0.63628604,  0.6257431,   0.61520571,  0.60467385,],
  [ 0.59414753,  0.58362676,  0.57311152,  0.56260183,  0.55209767,],
  [ 0.54159906,  0.53110598,  0.52061845,  0.51013645,  0.49966,   ]])
expected_m = np.asarray([
  [ 0.48,        0.49947368,  0.51894737,  0.53842105,  0.55789474],
  [ 0.57736842,  0.59684211,  0.61631579,  0.63578947,  0.65526316],
  [ 0.67473684,  0.69421053,  0.71368421,  0.73315789,  0.75263158],
  [ 0.77210526,  0.79157895,  0.81105263,  0.83052632,  0.85      ]])

# You should see relative errors around e-7 or less
print('next_w error: ', rel_error(expected_next_w, next_w))
```

```
print('v error: ', rel_error(expected_v, config['v']))
print('m error: ', rel_error(expected_m, config['m']))
```

```
next_w error:  1.1395691798535431e-07
v error:  4.208314038113071e-09
m error:  4.214963193114416e-09
```

Once you have debugged your RMSProp and Adam implementations, run the following to train a pair of deep networks using these new update rules:

In [12]:
```
learning_rates = {'rmsprop': 1e-4, 'adam': 1e-3}
for update_rule in ['adam', 'rmsprop']:
    print('Running with ', update_rule)
    model = FullyConnectedNet(
        [100, 100, 100, 100, 100],
        weight_scale=5e-2
    )
    solver = Solver(
        model,
        small_data,
        num_epochs=5,
        batch_size=100,
        update_rule=update_rule,
        optim_config={'learning_rate': learning_rates[update_rule]},
        verbose=True
    )
    solvers[update_rule] = solver
    solver.train()
    print()

fig, axes = plt.subplots(3, 1, figsize=(15, 15))

axes[0].set_title('Training loss')
axes[0].set_xlabel('Iteration')
axes[1].set_title('Training accuracy')
axes[1].set_xlabel('Epoch')
axes[2].set_title('Validation accuracy')
axes[2].set_xlabel('Epoch')

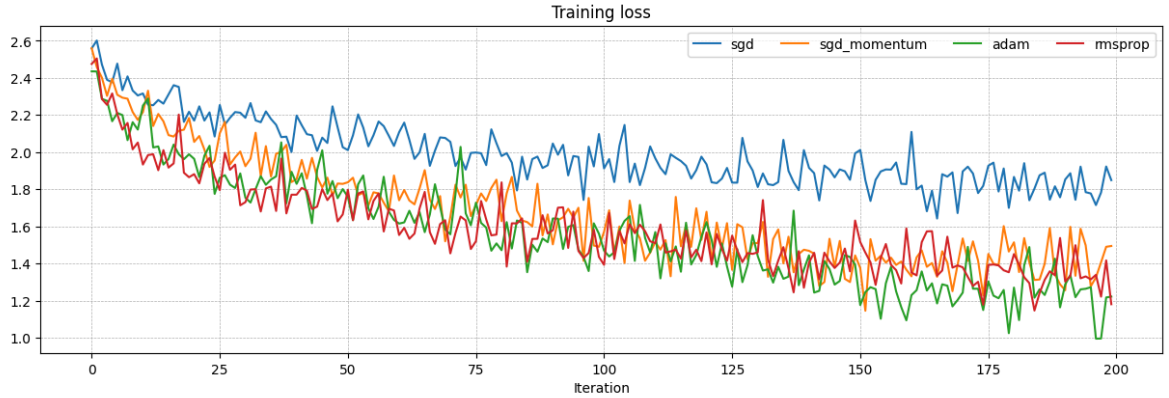
for update_rule, solver in solvers.items():
    axes[0].plot(solver.loss_history, label=f"{update_rule}")
    axes[1].plot(solver.train_acc_history, label=f"{update_rule}")
    axes[2].plot(solver.val_acc_history, label=f"{update_rule}")

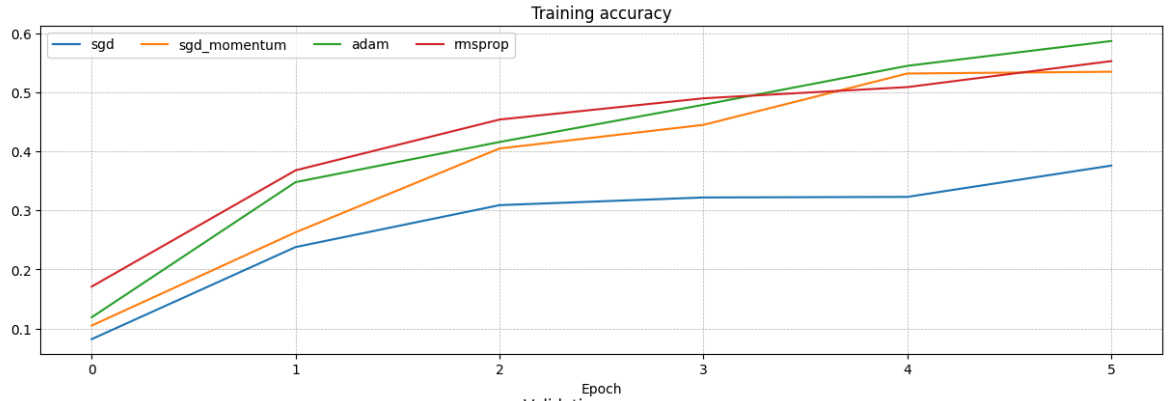
for ax in axes:
    ax.legend(loc='best', ncol=4)
    ax.grid(linestyle='--', linewidth=0.5)

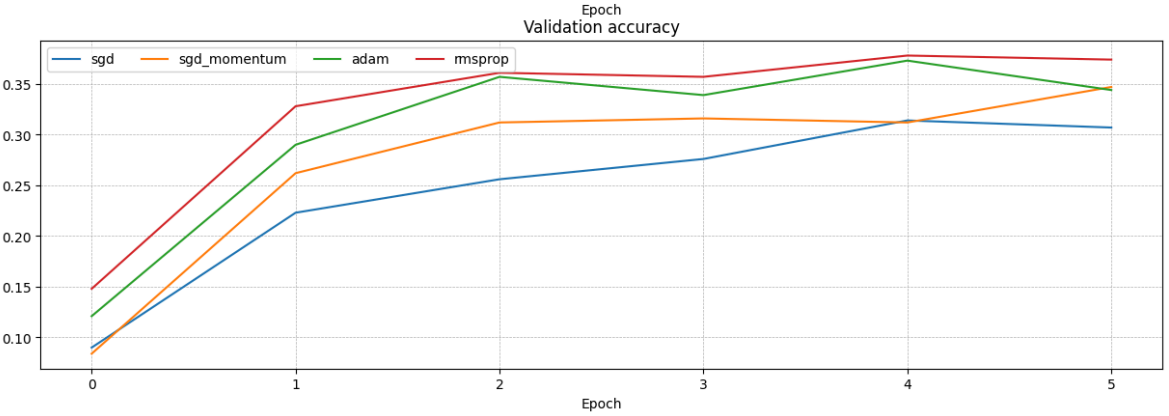
plt.show()
```

```
Running with  adam
(Iteration 1 / 200) loss: 2.435628
(Epoch 0 / 5) train acc: 0.119000; val_acc: 0.121000
(Iteration 11 / 200) loss: 2.251066
(Iteration 21 / 200) loss: 1.964071
(Iteration 31 / 200) loss: 1.754665
(Epoch 1 / 5) train acc: 0.348000; val_acc: 0.290000
(Iteration 41 / 200) loss: 1.829574
(Iteration 51 / 200) loss: 1.759403
(Iteration 61 / 200) loss: 1.615509
(Iteration 71 / 200) loss: 1.557887
(Epoch 2 / 5) train acc: 0.416000; val_acc: 0.357000
(Iteration 81 / 200) loss: 1.472111
(Iteration 91 / 200) loss: 1.642943
(Iteration 101 / 200) loss: 1.469783
(Iteration 111 / 200) loss: 1.569770
(Epoch 3 / 5) train acc: 0.479000; val_acc: 0.339000
(Iteration 121 / 200) loss: 1.623603
(Iteration 131 / 200) loss: 1.439919
(Iteration 141 / 200) loss: 1.444798
(Iteration 151 / 200) loss: 1.176649
(Epoch 4 / 5) train acc: 0.545000; val_acc: 0.373000
(Iteration 161 / 200) loss: 1.230187
(Iteration 171 / 200) loss: 1.243624
(Iteration 181 / 200) loss: 1.270742
(Iteration 191 / 200) loss: 1.287418
(Epoch 5 / 5) train acc: 0.587000; val_acc: 0.344000

Running with  rmsprop
(Iteration 1 / 200) loss: 2.475476
(Epoch 0 / 5) train acc: 0.171000; val_acc: 0.148000
(Iteration 11 / 200) loss: 1.933484
(Iteration 21 / 200) loss: 1.881700
(Iteration 31 / 200) loss: 1.730853
(Epoch 1 / 5) train acc: 0.368000; val_acc: 0.328000
(Iteration 41 / 200) loss: 1.771073
(Iteration 51 / 200) loss: 1.797652
(Iteration 61 / 200) loss: 1.553926
(Iteration 71 / 200) loss: 1.455002
(Epoch 2 / 5) train acc: 0.454000; val_acc: 0.361000
(Iteration 81 / 200) loss: 1.837432
(Iteration 91 / 200) loss: 1.579213
(Iteration 101 / 200) loss: 1.393301
(Iteration 111 / 200) loss: 1.510168
(Epoch 3 / 5) train acc: 0.490000; val_acc: 0.357000
(Iteration 121 / 200) loss: 1.567560
(Iteration 131 / 200) loss: 1.460583
(Iteration 141 / 200) loss: 1.405600
(Iteration 151 / 200) loss: 1.516222
(Epoch 4 / 5) train acc: 0.509000; val_acc: 0.378000
(Iteration 161 / 200) loss: 1.351827
(Iteration 171 / 200) loss: 1.379885
(Iteration 181 / 200) loss: 1.451219
(Iteration 191 / 200) loss: 1.303037
(Epoch 5 / 5) train acc: 0.553000; val_acc: 0.374000
```







Inline Question 2:

AdaGrad, like Adam, is a per-parameter optimization method that uses the following update rule:

```
cache += dw**2
w += - learning_rate * dw / (np.sqrt(cache) + eps)
```

John notices that when he was training a network with AdaGrad that the updates became very small, and that his network was learning slowly. Using your knowledge of the AdaGrad update rule, why do you think the updates would become very small? Would Adam have the same issue?

Answer:

AdaGrad 会不断累加历史梯度的平方，因此 cache 只增不减。随着训练进行，更新公式的分母逐渐增大，有效学习率越来越小，导致参数更新变小、学习变慢。 Adam 使用梯度平方的指数移动平均，历史梯度的影响会逐渐衰减，而不是一直累加。因此，它不会因为

AdaGrad 这种持续累积的机制而使有效学习率不断缩小，但仍可能因梯度较小或超参数设置不合适而学习缓慢。

Train a Good Model!

Train the best fully connected model that you can on CIFAR-10, storing your best model in the `best_model` variable. We require you to get at least 50% accuracy on the validation set using a fully connected network.

If you are careful it should be possible to get accuracies above 55%, but we don't require it for this part and won't assign extra credit for doing so. Later in the next assignment, we will ask you to train the best convolutional network that you can on CIFAR-10, and we would prefer that you spend your effort working on convolutional networks rather than fully connected networks.

Note: In the next assignment, you will learn techniques like BatchNormalization and Dropout which can help you train powerful models.

In [15]:
```
best_model = None

################################################################################
# TODO: Train the best FullyConnectedNet that you can on CIFAR-10. You might   #
# find batch/layer normalization and dropout useful. Store your best model in  #
# the best_model variable.                                                     #
################################################################################
# *****START OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
np.random.seed(231)

# 使用完整训练集
train_data = {
    'X_train': data['X_train'].astype(np.float32, copy=False),
    'y_train': data['y_train'],
    'X_val': data['X_val'].astype(np.float32, copy=False),
    'y_val': data['y_val'],
}

best_model = FullyConnectedNet(
    [256, 128],
    weight_scale=2e-2,
    reg=0.001,
    dtype=np.float32,
)

best_solver = Solver(
    best_model,
    train_data,
    update_rule='adam',
    optim_config={'learning_rate': 5e-4},
    lr_decay=0.95,
    num_epochs=30,
    batch_size=100,
    print_every=200,
    verbose=True,
)
```

```
best_solver.train()

print(f"最佳验证准确率: {best_solver.best_val_acc:.2%}")


# *****END OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
################################################################################
#                               END OF YOUR CODE                               #
################################################################################
```

```
(Iteration 1 / 14700) loss: 5.927978
(Epoch 0 / 30) train acc: 0.159000; val_acc: 0.164000
(Iteration 201 / 14700) loss: 1.821784
(Iteration 401 / 14700) loss: 1.802744
(Epoch 1 / 30) train acc: 0.495000; val_acc: 0.449000
(Iteration 601 / 14700) loss: 1.608711
(Iteration 801 / 14700) loss: 1.583179
(Epoch 2 / 30) train acc: 0.499000; val_acc: 0.460000
(Iteration 1001 / 14700) loss: 1.662223
(Iteration 1201 / 14700) loss: 1.728093
(Iteration 1401 / 14700) loss: 1.477298
(Epoch 3 / 30) train acc: 0.525000; val_acc: 0.490000
(Iteration 1601 / 14700) loss: 1.481389
(Iteration 1801 / 14700) loss: 1.124225
(Epoch 4 / 30) train acc: 0.551000; val_acc: 0.483000
(Iteration 2001 / 14700) loss: 1.366500
(Iteration 2201 / 14700) loss: 1.314175
(Iteration 2401 / 14700) loss: 1.405867
(Epoch 5 / 30) train acc: 0.570000; val_acc: 0.510000
(Iteration 2601 / 14700) loss: 1.460383
(Iteration 2801 / 14700) loss: 1.365270
(Epoch 6 / 30) train acc: 0.580000; val_acc: 0.517000
(Iteration 3001 / 14700) loss: 1.332351
(Iteration 3201 / 14700) loss: 1.352237
(Iteration 3401 / 14700) loss: 1.287892
(Epoch 7 / 30) train acc: 0.597000; val_acc: 0.520000
(Iteration 3601 / 14700) loss: 1.166214
(Iteration 3801 / 14700) loss: 1.231129
(Epoch 8 / 30) train acc: 0.633000; val_acc: 0.524000
(Iteration 4001 / 14700) loss: 1.142647
(Iteration 4201 / 14700) loss: 1.223642
(Iteration 4401 / 14700) loss: 1.109186
(Epoch 9 / 30) train acc: 0.624000; val_acc: 0.518000
(Iteration 4601 / 14700) loss: 1.273973
(Iteration 4801 / 14700) loss: 1.210482
(Epoch 10 / 30) train acc: 0.654000; val_acc: 0.506000
(Iteration 5001 / 14700) loss: 1.450527
(Iteration 5201 / 14700) loss: 1.172400
(Epoch 11 / 30) train acc: 0.664000; val_acc: 0.532000
(Iteration 5401 / 14700) loss: 1.272732
(Iteration 5601 / 14700) loss: 0.981282
(Iteration 5801 / 14700) loss: 1.155100
(Epoch 12 / 30) train acc: 0.689000; val_acc: 0.527000
(Iteration 6001 / 14700) loss: 0.940293
(Iteration 6201 / 14700) loss: 0.938714
(Epoch 13 / 30) train acc: 0.715000; val_acc: 0.542000
(Iteration 6401 / 14700) loss: 1.067307
(Iteration 6601 / 14700) loss: 1.172159
(Iteration 6801 / 14700) loss: 0.920731
(Epoch 14 / 30) train acc: 0.687000; val_acc: 0.493000
(Iteration 7001 / 14700) loss: 0.975894
(Iteration 7201 / 14700) loss: 0.897292
(Epoch 15 / 30) train acc: 0.701000; val_acc: 0.528000
(Iteration 7401 / 14700) loss: 0.991061
(Iteration 7601 / 14700) loss: 0.928682
(Iteration 7801 / 14700) loss: 0.817217
(Epoch 16 / 30) train acc: 0.712000; val_acc: 0.512000
(Iteration 8001 / 14700) loss: 0.863862
(Iteration 8201 / 14700) loss: 0.901816
(Epoch 17 / 30) train acc: 0.764000; val_acc: 0.517000
```

```
(Iteration 8401 / 14700) loss: 0.831658
(Iteration 8601 / 14700) loss: 0.673066
(Iteration 8801 / 14700) loss: 0.896869
(Epoch 18 / 30) train acc: 0.768000; val_acc: 0.510000
(Iteration 9001 / 14700) loss: 0.640452
(Iteration 9201 / 14700) loss: 0.964121
(Epoch 19 / 30) train acc: 0.780000; val_acc: 0.490000
(Iteration 9401 / 14700) loss: 0.743821
(Iteration 9601 / 14700) loss: 0.807932
(Epoch 20 / 30) train acc: 0.818000; val_acc: 0.528000
(Iteration 9801 / 14700) loss: 0.796482
(Iteration 10001 / 14700) loss: 0.660911
(Iteration 10201 / 14700) loss: 0.802508
(Epoch 21 / 30) train acc: 0.812000; val_acc: 0.522000
(Iteration 10401 / 14700) loss: 0.641104
(Iteration 10601 / 14700) loss: 0.642104
(Epoch 22 / 30) train acc: 0.794000; val_acc: 0.506000
(Iteration 10801 / 14700) loss: 0.734452
(Iteration 11001 / 14700) loss: 0.593992
(Iteration 11201 / 14700) loss: 0.597170
(Epoch 23 / 30) train acc: 0.813000; val_acc: 0.507000
(Iteration 11401 / 14700) loss: 0.646798
(Iteration 11601 / 14700) loss: 0.520723
(Epoch 24 / 30) train acc: 0.835000; val_acc: 0.522000
(Iteration 11801 / 14700) loss: 0.681689
(Iteration 12001 / 14700) loss: 0.596930
(Iteration 12201 / 14700) loss: 0.674435
(Epoch 25 / 30) train acc: 0.856000; val_acc: 0.507000
(Iteration 12401 / 14700) loss: 0.537667
(Iteration 12601 / 14700) loss: 0.621362
(Epoch 26 / 30) train acc: 0.845000; val_acc: 0.520000
(Iteration 12801 / 14700) loss: 0.562349
(Iteration 13001 / 14700) loss: 0.534796
(Iteration 13201 / 14700) loss: 0.610846
(Epoch 27 / 30) train acc: 0.867000; val_acc: 0.526000
(Iteration 13401 / 14700) loss: 0.569279
(Iteration 13601 / 14700) loss: 0.473263
(Epoch 28 / 30) train acc: 0.879000; val_acc: 0.515000
(Iteration 13801 / 14700) loss: 0.588302
(Iteration 14001 / 14700) loss: 0.493447
(Iteration 14201 / 14700) loss: 0.522056
(Epoch 29 / 30) train acc: 0.892000; val_acc: 0.517000
(Iteration 14401 / 14700) loss: 0.444232
(Iteration 14601 / 14700) loss: 0.509244
(Epoch 30 / 30) train acc: 0.890000; val_acc: 0.510000
最佳验证准确率：54.20%
```

Test Your Model!

Run your best model on the validation and test sets. You should achieve at least 50% accuracy on the validation set and the test set.

In [16]:
```
y_test_pred = np.argmax(best_model.loss(data['X_test']), axis=1)
y_val_pred = np.argmax(best_model.loss(data['X_val']), axis=1)
print('Validation set accuracy: ', (y_val_pred == data['y_val']).mean())
print('Test set accuracy: ', (y_test_pred == data['y_test']).mean())
```

```
Validation set accuracy:  0.542
Test set accuracy:  0.502
```

In [ ]: